



MIT Art, Design and Technology University MIT School of Computing, Pune

Department of Information Technology

Lab Manual

Practical - Web Programming

Class - S.Y. (SEM-II), DA

Batch - DA-I

Ms. Diya Patil

A.Y. 2025 – 2026 (SEM-II)

Web Programming SEMESTER – IV			
Course Code:	23IT2008	Course Credits:	02
Teaching Hours / Week (L:T:P):	0:0:4	CA Marks:	25
Total Number of Teaching Hours:		END-SEM Marks:	25
Course Pre-requisites:			
Course Description: This course provides a comprehensive introduction to web technology, designed to help students develop a strong foundation in building and managing websites and web applications. The curriculum covers key topics such as HTML, CSS, and JavaScript, PHP, MySQL, which are essential for creating interactive, well-designed web pages. Students will also explore the principles of responsive design, ensuring that web applications are optimized for different devices and screen sizes. The course dives deeper into server-side technologies, including HTTP, web servers, and databases, allowing students to understand how websites function behind the scenes. Emphasis is placed on practical learning, and students will gain hands-on experience by working on projects that showcase their ability to design, develop, and deploy websites. By the end of the course, students will be proficient in using modern web technologies to create web applications. They will understand how to handle client-server interactions, manage user data, and implement various web technologies to enhance the functionality of their applications.			
Course Learning Objectives: This course will enable the students to: 1. Understand fundamental concepts of front-end web development. 2. Enable students to create basic web pages incorporating essential elements such as images, hyperlinks, lists, tables, and forms. 3. Teach students how to use CSS to manage fonts, lists, colors, text alignment, and background images for a cohesive and aesthetically pleasing web design. 4. Develop an understanding of JavaScript scopes to manage the visibility and lifetime of variables and functions effectively. 5. Equip students with the skills to implement and handle JavaScript events, enabling enhanced user interactions through event-driven programming. 6. Apply comprehensive knowledge of HTML, CSS, and JavaScript to develop a complete front-end application. Utilize project-based learning to showcase problem-solving skills and creativity in web development projects. 7. Configure server environments with Apache/TOMCAT. 8. Set up a PHP development environment and write basic PHP scripts. 9. Master PHP programming constructs for web development tasks. 10. Create and process HTML forms, and manage MySQL database operations. 11. Develop comprehensive back-end applications using PHP and MySQL.			
Course Outcome: After taking this course, Students will be able to : 1. Apply knowledge of HTML to create the structure of the webpage and CSS to style and layout the elements, making the application visually appealing. 2. Apply comprehensive knowledge of HTML, CSS, and JavaScript to develop a complete front-end application and utilize project-based learning to showcase problem-solving skills and creativity in web development projects. 3. Set up and configure a server environment using tools like Apache or TOMCAT and set up a PHP development environment. Write & execute simple PHP scripts, understanding PHP syntax and basic features, create HTML forms to collect user data and integrate with PHP			

for processing.

4. Design and develop a back-end application using PHP and MySQL, implementing CRUD operations to manage data effectively.

UNIT - I	Introduction to HTML and Cascading Style Sheet	09 Hours
Module 1 - Markup Language (HTML): Introduction to HTML, Formatting and Fonts, Commenting Code, Anchors, Backgrounds, Images, Hyperlinks, Lists, Tables, Frames, HTML Forms Module 2 - CSS: Need for CSS, introduction to CSS, basic syntax and structure, Levels of style sheets, Style specification formats, BOX Model, Selector forms, Property value forms, Font properties, List properties, Color, Alignment of text, Background images		
Pedagogy	ICT Teaching / PowerPoint Presentation and Videos: Use tools like Visual Studio Code (free). Videos: https://www.coursera.org/learn/html-css-javascript-for-web-developers	
	Self-study / Do it yourself /: Practice creating basic HTML pages and enhancing them using CSS.	
	Experiential Learning Topics: Design a simple webpage for Fragrance shop website	
	PBL - Project Based Learning: Create a multi-page website (e.g., Fragrance shop website) using HTML and CSS.	
UNIT - II	Front-End Development	09 Hours
Module 3 - Overview of JavaScript, including JS in an HTML (Embedded, External), Basic JS syntax, basic interaction with HTML Module 4 - Core features of JavaScript: Data types, Control Structures, Arrays, Functions and Scopes		
Pedagogy	ICT Teaching / PowerPoint Presentation and Videos: Use tools like Visual Studio Code (free). Videos: https://www.coursera.org/learn/javascript-basics	
	Self-study / Do it yourself /: Solve exercises on JavaScript syntax, control structures, and functions	
	Experiential Learning Topics: Build a web page with interactive elements (e.g., a simple calculator).	
	PBL - Project Based Learning: Develop an interactive webpage that uses JavaScript to validate form inputs or perform basic calculations.	
UNIT - III	Advanced Front-End Development	09 Hours
Module 5 - DOM: DOM levels, DOM Objects and their properties and methods, Manipulating DOM Module 6 - JavaScript Events: JavaScript Events, Types of JavaScript Events, Objects in JS, Event Handling		
Pedagogy	ICT Teaching / PowerPoint Presentation and Videos: https://www.coursera.org/learn/building-interactive-web-pages-using-javascript Use tools like Visual Studio Code (free).	

	Self-study / Do it yourself /: Practice exercises on DOM traversal and event handling.	
	Experiential Learning Topics: Add dynamic behavior to a webpage using DOM and events (e.g., a to-do list app).	
	PBL - Project Based Learning: Develop a web page with dynamic content (e.g., a task manager or interactive quiz) using DOM manipulation and event handling.	
UNIT – IV	Server Side Scripting	09 Hours
Module 7 - Set up and configure a server environment using tools like Apache or TOMCAT, set up a PHP development environment. Module 8 -Introduction to PHP: : Introduction to PHP, Server side scripting Vs Client side scripting, Basic Development Concepts (Mixing PHP with HTML), Creating, Writing & Running First PHP Script, PHP syntax, conditions & Loops, Functions, String manipulation, Arrays & Functions, Module 9 - Form handling with HTML and PHP: Designing of Forms using HTML, Form Handling using GET and POST methods of Form		
Pedagogy	ICT Teaching / PowerPoint Presentation and Videos: https://www.coursera.org/learn/web-applications-php Use tools like Visual Studio Code (free), XAMPP/WAMP for PHP server setup, and MySQL Workbench for database management	
	Self-study / Do it yourself /: Practice exercises on form handling and server-side scripting with PHP.	
	Experiential Learning Topics: Create a basic form for data submission and handle it using PHP (e.g., feedback form).	
	PBL - Project Based Learning: Develop a small server-side application (e.g., a contact form with email validation and submission).	
UNIT – V	Working with Databases and Web Application Development	09 Hours
Module 10 - Working with databases using MySQL with PHP: MySQL database, create database, create table, primary key with AUTO_INCREMENT setting, Insert Data Into a Database Table, Select Data From a Database Table, Open or close a Connection to the MySQL Server. Module 11 - Web Application Development (Project): Develop the web application to handle client-server interactions, manage user data, and implement various web technologies to enhance the functionality of their applications. Example: Website for a Fragrance shop		
Pedagogy	ICT Teaching / PowerPoint Presentation and Videos: Use tools like Visual Studio Code (free), XAMPP/WAMP for PHP server setup, and MySQL Workbench for database management Videos: https://www.coursera.org/learn/web-app	
	Self-study / Do it yourself /: Exercises on creating and manipulating databases using PHP and MySQL.	
	Experiential Learning Topics:	

	Create a database and design a webpage to display its data dynamically. PBL - Project Based Learning: Develop a fully functional web application (e.g., a Fragrance shop website or e-commerce platform) that integrates database functionality for data management.
--	--

Text Books:

1. "HTML and CSS: Design and Build Websites" by Jon Duckett.
2. "Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics" by Jennifer Niederst Robbins.
3. Achyut Godbole & Atul Kahate, |Web Technologies: TCP/IP to Internet Application Architectures|, McGraw Hill Education publications, ISBN, 007047298X, 9780070472983.
4. Ralph Moseley & M. T. Savaliya, —Developing Web Applications|, Wiley publications, ISBN 13 : 9788126538676.

Reference Books:

1. Eloquent JavaScript: A Modern Introduction to Programming by Marijn Haverbeke.
2. JavaScript: The Good Parts by Douglas Crockford.
3. CSS Secrets: Better Solutions to Everyday Web Design Problems by Lea Ver.
4. Web Technologies- Jeffery C. Jackson, ISBN 978-81-317-1715-8 Pearson 2015.
5. PHP Objects, Patterns, and Practice by Matt Zandstra
6. MySQL Cookbook by Paul DuBois.
7. Advanced PHP Programming - George Schlossnagle- ISBN 0-672-32561-6,2004.

URLs (Optional) - List of Online Courses

1. W3Schools HTML, CSS, JavaScript Tutorial: <https://www.w3schools.com/html/>
2. Mozilla Developer Network (MDN) Web Docs - HTML, CSS, JavaScript, DOM: https://developer.mozilla.org/en-US/docs/Learn/HTML/Introduction_to_HTML
3. Project-Based Learning Resources: <https://developer.mozilla.org/en-US/docs/Learn>

Contents beyond Syllabus:

1. Web Essentials
2. Using JavaScript to handle form submission and login events (e.g., onsubmit, onclick)
3. JavaScript Form validations, General Input Validation, Password Validation
4. Storing user data (like a username) temporarily using localStorage or sessionStorage
5. Dynamically updating the content of the webpage, such as displaying a welcome message
6. Redirecting users using window.location

List of Experiments:

In this series of assignments, you will create a Fragrance shop / any other website step by step. Each assignment will focus on a different aspect of the website, covering various HTML elements, CSS, JavaScript, PHP and MySQL concepts.

Laboratory/Project Assignment Guidelines:

1. Project Selection:

- Each student must select a unique project topic for their laboratory assignments.
- The chosen project topic should align with the concepts covered in the course syllabus.
- The chosen project topic should be approved by the course coordinator/ subject teacher.
- Students have the freedom to choose their project topics based on their interests and career aspirations.
- Project topics may include but are not limited to:
 - E-commerce website
 - Blogging platform
 - Online booking system
 - Content management system (CMS)
 - Discussion forum
 - Social networking platform
 - Task management application
 - Portfolio website

2. Laboratory Assignments:

- Throughout the course, students will complete laboratory assignments related to their chosen project topic.

3. Evaluation Criteria:

- The laboratory assignments and the final project will be evaluated based on criteria such as Structure and Semantics, Content Organization, Forms and Inputs, Links and Navigation, Styling and Layout, Design Consistency, Functionality, Code Quality and adherence to project requirements.
- Students are expected to demonstrate creativity, and a comprehensive understanding of web development principles in their projects.
- The laboratory assignments based on chosen project topics will be assessed based on several key criteria that reflect both technical proficiency and creative application in web development. These include:
 - Structure & Semantics: Proper use of HTML to create a logical, accessible structure with meaningful and semantically correct elements.
 - Content Organization: Clear and intuitive organization of content, ensuring ease of navigation and logical flow throughout the site.
 - Forms & User Input: Effective implementation of forms and user input elements that are functional, validated, and accessible.
 - Links & Navigation: Well-structured navigation and functional links that provide a seamless user experience.
 - Styling & Layout: Visually appealing and responsive design, with a well-executed layout that adapts to various screen sizes.
 - Design Consistency: Uniformity in design elements, including colors, typography, and spacing, to maintain a cohesive look and feel across the site.

- **Functionality:** Full functionality of all interactive elements, ensuring a bug-free, smooth experience for users.
- **Code Quality & Best Practices:** Clean, well-organized, and efficient code that adheres to modern web development best practices and is easy to maintain.

4. Submission and Presentation:

- The project and project report/journal must be submitted within the specified deadline and should meet the specified requirements outlined by the course coordinator/ subject teacher.

Project Problem Statement-

Design and develop a basic website for a local Fragrance shop using HTML, CSS, JavaScript, PHP and MySQL. This website will serve as an online presence for the Fragrance shop, effectively communicating the brand identity, showcasing the menu, providing essential information, and allowing customers to easily get in touch or locate the shop.

The project directory is as follows:

coffee-shop-website/

```

├── css/
│   └── styles.css
├── html/
│   ├── index.html
│   ├── menu.html
│   ├── about.html
│   └── contact.html
├── js/
│   └── scripts.js
└── images/

```

1.	Create the basic structure of the fragrance shop website, including the home page layout with a header, main content area, and footer. Prepare a common project website design and plan document for all assignments. Consider following points: 1. Brief information about the project. 2. Set the goals & deliverables. 3. Finalize the modules of the project. 4. Define the audience. 5. Describe pain points & the ideal experience (On the basis of existing systems) 6. Set the visual direction
----	--

	7. Map out the Project structure. 8. Plan the content for each page. 9. Add ideas for content, images & layout. 10. Determine your site structure or Create content for your core website pages: a. Home page b. About page c. Product/Service page d. Testimonial/review page e. Contact page f. Starter blog posts 11. Create and collect design elements These design elements define your brand personality and help customers feel what your brand represents through the use of: a. Colors b. Fonts and typography c. Logos d. Images and photos
2.	HTML ● Create a detailed home page for the Fragrance shop website. ● Create a detailed menu/product page for the Fragrance shop website, listing all available items categorized appropriately. ● Create a cart page that allows customers to review and manage the items they wish to purchase before proceeding to checkout. ● Create an about us page that provides detailed information about the Fragrance shop's history, mission, and team. ● Create a contact page that allows customers to easily get in touch with the Fragrance shop through a form. ● Design and implement admin/user registration form for the Fragrance shop website. ● Design and implement admin/user login form for the Fragrance shop website.
3.	CSS ● Enhance the layout of the Fragrance shop website using CSS Grid for the home page. ● Use CSS Grid to layout the menu/product items in a structured and style the menu categories with appropriate headings, spacing, separators, images, descriptions, and prices.

4.	CSS - Enhance the cart page to make it user-friendly and visually appealing. Style the cart items with appropriate margins, paddings, and input field styles to provide a seamless shopping experience. - Enhance and style the about us page with appropriate margins, paddings, and input field styles. - Enhance and style the contact page to make it user-friendly and visually appealing. Style the contact form with appropriate margins, paddings, and input field styles. - Enhance and style the admin/user registration form with appropriate margins, paddings, and input field styles. - Enhance and style the admin/user login form with appropriate margins, paddings, and input field styles.
5.	JavaScript - Implement user registration and login forms for the Fragrance shop website. These forms will allow users to create an account, log in, and access personalized features, such as saving favorite items or viewing order history. - User Registration Form will allow new customers to sign up and create an account on the website. The form will capture basic user details, including the name, email address, and password (not limited to these fields). - User Login Form will allow registered users to log into their accounts. The form will require an email address and a password to authenticate the user. - Provide validations for user registration and login forms to validate the input to ensure that all required fields are filled and that the email format is valid. (Contents beyond Syllabus) - Develop cart functionality to allow users to add items, update quantities, and remove items.
6.	JavaScript - The user login form will allow registered users to log into their accounts. The form will require an email address and a password to authenticate the user. - If the login is successful, the user should be redirected to the homepage or their user dashboard. (Contents beyond Syllabus) - Use localStorage or sessionStorage to store authentication data, such as the user's email and login status. This ensures that once a user is logged in, they remain authenticated even after the page reloads or when they visit the site again. (Contents beyond Syllabus) - Save the cart data to local storage when items are added, updated, or removed. Retrieve and load the cart data from local storage when the page loads. (Contents beyond Syllabus)
7.	PHP

	• Develop a PHP script to handle user registration for the Fragrance shop website. The script should accept input from users for their name, email address, password, etc. (all required fields for registration). • Implement error handling to notify users of any issues during registration, such as validation errors. • Provide feedback to the user upon successful registration, either through a confirmation message or a redirect to a login page.
8.	PHP • Develop a PHP script to handle user login for the Fragrance shop website. The script should accept input from users for their login credentials. (all required fields for login). • Provide feedback to the user upon successful login, either through a confirmation message or a redirect to a welcome page. • Implement error handling to notify users of login failures due to incorrect credentials or other errors. • Provide feedback to the user upon successful login, either through a welcome user message or a redirect to a home page.
9.	PHP and MySQL • Develop a PHP script that allows users to manage their shopping cart for an e-commerce website (e.g., a Fragrance shop store). The script should allow users to add items to their cart, view their cart contents, and remove items if needed. • Develop a PHP script to manage the shopping cart for an e-commerce website (e.g., a Fragrance shop store) using MySQL. This script should allow users to add items to their cart, view their cart contents, and remove items from the cart. The cart data should be stored in the MySQL database to allow persistence across sessions.
10.	PHP and MySQL • Develop a PHP script to handle the checkout process for users who are ready to complete their purchase. The script should process the cart data and provide feedback to the user upon successful or failed checkout. • Develop a PHP script that processes the checkout process for users who are ready to complete their purchase, integrating the MySQL database for handling user and order information. The script should validate user input, process the cart data, and provide feedback upon successful or failed checkout.

Experiment No.1

Problem Statement:

1. Create the basic structure of the Fragrance shop website, including the home page layout with a header, main content area, and footer.

Prepare a common project website design and plan document for all assignments. Consider following points:

1. Brief information about the project.
2. Set the goals & deliverables.
3. Finalize the modules of the project.
4. Define the audience.
5. Describe pain points & the ideal experience (On the basis of existing systems)
6. Set the visual direction
7. Map out the Project structure.
8. Plan the content for each page.
9. Add ideas for content, images & layout.
10. Determine your site structure or Create content for your core website pages:
 - a. Home page
 - b. About page
 - c. Product/Service page
 - d. Testimonial/review page
 - e. Contact page
 - f. Starter blog posts
11. Create and collect design elements
12. These design elements define your brand personality and help customers feel what your brand represents through the use of:
 - a. Colors
 - b. Fonts and typography
 - c. Logos
 - d. Images and photos

Objective:

To design the basic structure of a Fragrance shop website by planning its layout, content, and visual elements, ensuring it meets user needs and effectively represents the brand.

Theory:

Project Design and Plan Document for Fragrance shop Website

1. Brief Information about the Project

The project is to create a user-friendly and visually appealing website for a Fragrance shop. It aims to attract coffee enthusiasts, highlight products, and offer features such as customer testimonials and a contact platform. Additionally, the website will support login and registration to personalize user experiences and allow secure access to exclusive features.

2. Goals and Deliverables

Goals

- Develop an engaging and functional website for a Fragrance shop.
- Showcase the shop's story, products, customer reviews, and contact details.

- Enable users to register, log in, and personalize their experience.
- Create a responsive website that works across all devices.

Deliverables

- Website Pages:
 - Home Page
 - About Page
 - Products/Services Page
 - Testimonials Page
 - Contact Page
 - Login Page
 - Registration Page
 - Starter blog posts or placeholder for future blogs (optional).
- Core Features:
 - Header and footer with consistent navigation.
 - Functional login and registration system.
 - Responsive design adaptable to mobile, tablet, and desktop.
 - Professional design with appropriate use of colors, fonts, and images.

3. Finalize the modules of the project

The Fragrance shop website will have a modular structure that ensures easy navigation, usability, and maintenance. Each module corresponds to a distinct functionality or page, helping in modular development and integration. Below is a detailed description of the finalized modules:

Website Modules

1. Home Page Module

- Description:

The main page of the website welcomes users and highlights essential features. It sets the tone for the user experience.
- Features:
 - Hero section with the tagline and call-to-action buttons (e.g., "Order Now" or "Explore Menu").
 - Overview of featured products or promotions.
 - Navigation menu linking to all website sections (e.g., About, Products, Testimonials, Contact, Login).
 - Footer with contact details, social links, and other information.

2. About Page Module

- Description:

Offers visitors a glimpse of the Fragrance shop's story, mission, and values.
- Features:
 - Introduction to the Fragrance shop's history and team.
 - Showcase the brand's principles like quality, sustainability, and customer service.
 - Engaging visuals to reflect the Fragrance shop's vibe.

3. Products/Services Page Module

- Description:
Displays the shop's product offerings in a user-friendly way.
- Features:
 - Categorized menu (e.g., Coffee, Snacks, Beverages).
 - Images and details for each item, including price and description.
 - Option for filtering or searching products (future enhancement).

4. Testimonials Page Module

- Description:
Shares positive customer reviews and builds trust with new visitors.
- Features:
 - Slider or grid layout showcasing testimonials.
 - Include a field or section for customers to submit their reviews (optional).

5. Contact Page Module

- Description:
Enables visitors to get in touch with the Fragrance shop.
- Features:
 - A form for user inquiries (fields: Name, Email, Subject, Message).
 - Embedded map for the physical shop location.
 - Display contact details like phone number and working hours.

6. Login Page Module

- Description:
Provides authentication functionality for returning users.
- Features:
 - Login form with fields for Email and Password.
 - "Forgot Password?" link.
 - Redirection to the registration page for new users.

7. Registration Page Module

- Description:
Allows new users to sign up for an account.
- Features:
 - Registration form with fields for Name, Email, and Password creation.
 - Terms and conditions acceptance checkbox.
 - Submit button to create an account.

8. Footer Module

- Description:
A common footer displayed across all pages.
- Features:
 - Links to Privacy Policy, Terms of Service, and social media pages.
 - Address and basic contact info.

4. Define the audience

Target Audience

The website for the Fragrance shop is designed to cater to a broad spectrum of visitors, each with specific needs and expectations. Understanding the audience ensures the website design, content, and features meet their requirements. Below is a breakdown of the target audience:

a. Coffee Enthusiasts

- Characteristics:
 - Regular consumers of coffee.
 - Interested in exploring various coffee flavors, brewing methods, and exclusive products.
- Needs:
 - Detailed product descriptions with images and pricing.
 - Access to exclusive offers, rewards, or subscriptions for frequent purchases.

b. Professionals and Remote Workers

- Characteristics:
 - Need a quiet, comfortable space to work or meet clients.
 - May order coffee and snacks online or reserve seating.
- Needs:
 - Contact page with opening hours and location details.
 - Online ordering system for in-store pickups or delivery.
 - Wi-Fi information or co-working amenities.

c. Students

- Characteristics:
 - Younger audience looking for affordable and quick snacks, beverages, and a social hangout space.
- Needs:
 - Discounts for students or groups.
 - Easy navigation to view menus and promotional deals.

d. Health-Conscious Customers

- Characteristics:
 - Focused on dietary requirements, organic options, or alternative milk and low-calorie beverages.
- Needs:
 - Highlight nutritional information on the menu.
 - Clear categorization of vegan or healthy options.

e. Tourists and Travelers

- Characteristics:
 - Look for local cafes offering an authentic experience.
 - Interested in souvenirs, gift cards, and specialty items.
- Needs:
 - Integration with map navigation to locate the cafe.
 - Details about specialty drinks or regional favorites.

f. New Users (Unfamiliar Customers)

- Characteristics:
 - New visitors who haven't interacted with the brand before.
- Needs:
 - A professional and appealing "About Us" page to share the shop's values and story.
 - A clean, user-friendly interface to build trust.

g. Online Shoppers

- Characteristics:
 - Prefer the convenience of ordering coffee products online, including beans and brewing equipment.
- Needs:
 - Product page with a seamless e-commerce experience.
 - Secure login and registration system for repeat shopping.

Website Features Mapped to Audience Needs:

Audience Segment	Key Features Needed
Coffee Enthusiasts	Menu page with detailed descriptions and coffee brewing tips.
Professionals/Remote Workers	Online ordering and pickup, clear navigation to amenities, contact page with location and hours.
Students	Discounts, loyalty programs, or group offers listed prominently.
Health-Conscious Customers	Categorized menu with nutritional facts and health-oriented filters.
Tourists/Travelers	Geolocation features, unique content promoting local specialties.
New Users	Intuitive UI/UX design with clear site navigation and testimonials to build credibility.
Online Shoppers	Secure login and product pages with clear categories for beans, equipment, or coffee subscriptions.

Why Understanding the Audience is Important

- Helps in creating engaging and relevant content tailored to users' preferences.
- Enhances the user experience (UX) by addressing specific pain points and ensuring seamless navigation.
- Builds brand trust and attracts loyal customers who resonate with the Fragrance shop's story and mission.
- Leads to targeted marketing campaigns, such as student promotions, subscription offers for enthusiasts, or health-focused messaging.

5. Describe pain points & the ideal experience (On the basis of existing systems)

1. Identifying Pain Points of Existing Systems

a. Pain Point: Poor Navigation and Cluttered Interface

- Issue: Many Fragrance shop websites have complicated or cluttered designs that make it hard for users to find what they are looking for.
- Impact: Users often leave the site due to frustration or lack of usability.

b. Pain Point: Limited Online Ordering Functionality

- Issue: Existing systems often do not provide easy-to-use online ordering features, resulting in lower conversion rates and fewer sales.
- Impact: Loss of potential customers who prefer the convenience of online purchases.

c. Pain Point: Lack of Mobile Optimization

- Issue: Non-responsive designs lead to poor mobile user experience.
- Impact: Customers using smartphones face issues navigating the site, viewing products, or ordering items.

d. Pain Point: Insufficient Product Information

- Issue: Customers do not get enough details about menu items, including ingredients, dietary considerations, and prices.
- Impact: Potential customers abandon their search due to incomplete information.

e. Pain Point: Weak Engagement Strategies

- Issue: Existing websites lack features like loyalty programs, student discounts, or engaging content like blogs.
- Impact: Missed opportunities for creating brand loyalty and retaining customers.

f. Pain Point: Inefficient Contact and Location Details

- Issue: Many websites fail to prominently display contact and location information, making it difficult for customers to find or connect with the Fragrance shop.
- Impact: Customers waste time searching and may opt for competitors instead.

g. Pain Point: No Personalization Options

- Issue: The lack of personalized user experiences or features like accounts, favorite orders, or personalized recommendations.
- Impact: Users feel the service is impersonal, leading to decreased satisfaction.

2. Crafting the Ideal Experience

To address these pain points, the website design and functionality should create a user-friendly, visually appealing, and highly interactive experience.

a. Intuitive Navigation and Clean Design

- Use a clear and consistent layout with a sticky navigation bar.
- Include links to all key pages (Home, About, Menu, Testimonials, Contact, Login/Sign Up).

b. Seamless Online Ordering

- Implement a robust e-commerce system allowing customers to browse products, add items to a cart, and complete purchases effortlessly.
- Provide features like "Order Now" buttons on the homepage and menu pages.

c. Mobile-Responsive Design

- Design with a mobile-first approach, ensuring compatibility across devices.

- Use flexible grids, touch-friendly elements, and optimized performance for fast loading times.

d. Comprehensive Product Information

- Include high-quality images, item descriptions, ingredients, prices, and allergy/dietary labels (e.g., gluten-free or vegan).
- Create filters for health-conscious customers, like "Low Calorie" or "Vegan Options."

e. Customer Engagement Features

- Introduce loyalty programs with a points system visible after login.
- Offer a blog with content like coffee brewing tips, health benefits, or shop news.
- Highlight customer reviews and testimonials on a dedicated page.

f. Easy Access to Contact and Location

- Include a contact page with a simple inquiry form, phone number, and email.
- Display an embedded map for the shop's physical location on the homepage or contact page.

g. Personalization

- Allow users to create accounts for saving their favorite items or past orders.
- Use a welcome message with the customer's name after login.
- Send personalized offers via email for registered users.

3. The Ideal User Journey

Step 1: Visiting the Website

- Users arrive at a welcoming homepage with clear navigation to different sections.

Step 2: Browsing the Menu

- Users navigate to the products/services page, view clear menus, and filter items based on preferences.

Step 3: Placing an Order

- Users can seamlessly add items to their cart and complete a purchase with minimal clicks.

Step 4: Finding Location or Contacting Support

- Users easily locate contact and location details for in-store visits or inquiries.

Step 5: Engaging with Content

- Users read blogs or testimonials for a deeper connection with the brand.

Step 6: Creating Loyalty

- Registered users receive personalized promotions or gain points through purchases.

6. Set the visual direction

1. Visual Design Goals

The visual design of the Fragrance shop website should reflect its personality, build trust, and create an inviting experience for customers. It should align with the following principles:

- **Welcoming and Comfortable:** The website should feel cozy and approachable, much like the ambiance of the Fragrance shop itself.
- **Modern and Minimalistic:** Clean layouts and modern design elements create a professional and user-friendly aesthetic.
- **Brand Representation:** The visual elements, including colors, typography, and images, should communicate the Fragrance shop's values and target audience.

2. Defining the Core Visual Elements

a. Color Palette

A warm and earthy color palette inspired by coffee and natural tones creates a visually consistent and soothing experience.

Color	Hex Code	Usage
Coffee Brown	#6F4E37	Header, footer, buttons, and highlights.
Creamy Beige	#F5F5DC	Background to create warmth and contrast.
Deep Espresso	#3C2F2F	Text and important accents for legibility.
Latte White	#FAF3E0	Secondary backgrounds and subtle contrasts.
Olive Green	#556B2F	Call-to-action buttons for natural harmony.

b. Typography

Fonts should be easy to read while reflecting the warm and inviting atmosphere of the Fragrance shop.

- Primary Font: *Poppins* or *Roboto* (Sans-serif) – For headings and call-to-action text.
- Secondary Font: *Open Sans* or *Lora* – For body text and descriptions.
- Attributes: Use bold headings for emphasis and lighter weights for readability.

c. Logos and Branding

A sleek, memorable logo based on the coffee theme is essential. For instance:

- Use a stylized coffee cup, bean, or steam motif in the logo design.
- The logo should include the Fragrance shop's name in the selected typography.
- A monochrome version of the logo can be created for simplicity in headers or footers.

d. Imagery and Icons

High-quality visuals can make the website feel alive and inviting.

- Photography:
 - Pictures of freshly brewed coffee, beans, cozy seating spaces, and happy customers.
 - Showcase specialty coffee drinks, desserts, and in-store ambiance.
- Icons:
 - Minimalistic icons for categories like menu, location, testimonials, and contact.
- Hero Images:
 - Use a carousel or static hero banner on the homepage featuring key products or seasonal promotions.

3. Applying Visual Design to Pages

a. Home Page

- Banner Area: Hero image with text overlay showcasing a featured product or promo.
- Color Scheme: Warm tones for text and buttons, cream or latte shades for the background.
- Typography: Bold for headlines like "Welcome to [Fragrance shop Name]."

b. About Page

- Use authentic imagery of staff, customers, or the coffee preparation process.
- Soft, inviting colors to match the tone of storytelling.

c. Product/Service Page

- Ensure product cards display item images prominently with prices and descriptions.
- Add hover effects to highlight product interactions.

d. Testimonial Page

- Use customer photos and quotes with clean card layouts.
- A slider element for seamless scrolling through reviews.

e. Contact Page

- Embed Google Maps with the shop's location using the green-toned call-to-action buttons.

f. Login and Registration Pages

- Keep the form layout minimal, with soft color backgrounds and visible input fields.
- Buttons in coffee brown or green for consistent branding.

4. Layout and Design Hierarchy

The visual hierarchy ensures an easy and intuitive flow through the website:

1. Headers and Banners: Prominent for branding and immediate engagement.
2. Navigation Bar: Sticky and unobtrusive for easy exploration.
3. Sections and Grids: Structured with clear breaks using background shades or borders.
4. Call-to-Action: Buttons prominently styled to encourage actions like "Order Now" or "Sign Up."

5. Expected Impact of Visual Direction

1. Enhanced Engagement: A warm design encourages users to explore further.
2. Stronger Branding: Consistency in colors and typography strengthens identity.
3. Better Retention: User-friendly layouts and aesthetic appeal retain visitors.
4. Higher Conversions: Effective call-to-action placement drives purchases or registrations.

7. Map out the Project structure

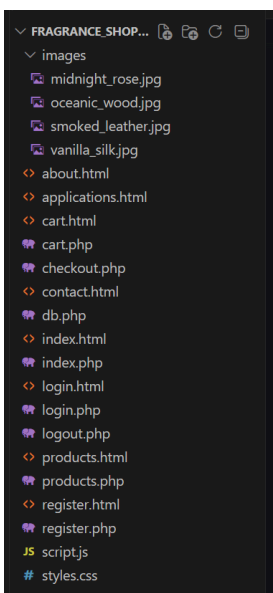
Fragrance_shop_website/

```

|
|— images/           # Folder containing all your product photos
| |— midnight_rose.jpg
| |— oceanic_wood.jpg
| |— smoked_leather.jpg

```

	└─ vanilla_silk.jpg	
	└─ index.php	# Dynamic Home page
	└─ about.html	# Static About Us page
	└─ applications.html	# Static Applications page
	└─ products.php	# Dynamic Shop/Collection page
	└─ cart.php	# Dynamic Shopping cart page
	└─ checkout.php	# Dynamic Order processing logic
	└─ contact.html	# Static Contact page
	└─ login.php	# Dynamic User login
	└─ register.php	# Dynamic User account creation
	└─ logout.php	# Dynamic Session termination
	└─ db.php	# Database connection configuration
	└─ script.js	# Updated JavaScript (without the old interception code)
	└─ styles.css	# Global luxury stylesheet



8. Plan the content for each page

The website will include a minimum of 5 core pages, along with additional Login and Registration pages. This plan details the content for each page.

1. Home Page

Purpose:

- Welcome visitors.
- Highlight the Fragrance shop's offerings and features.

Content Plan:

- Header:
 - Logo on the left.
 - Navigation menu: Home, About, Products, Testimonials, Contact.
 - Login/Sign-Up button on the top right.

- Hero Section:
 - A high-quality banner image (Fragrance shop ambiance, a featured drink).
 - A tagline like: *"Brewed to Perfection!"*.
 - CTA button: *"Explore Our Menu"*.
- Introduction Section:
 - Brief about the shop (one or two sentences).
 - CTA: *"Learn More About Us"* linking to the About page.
- Special Offer/Highlight Section:
 - Carousel or grid of featured drinks or offers.
 - Text: *"Try Our New Seasonal Flavors!"*.
- Footer:
 - Quick links, social media links, contact information.

2. About Page

Purpose:

- Share the story, vision, and people behind the Fragrance shop.

Content Plan:

- Header: (same as home page).
- About Us Section:
 - A short introduction to the Fragrance shop's history (e.g., when and why it was founded).
 - Emphasis on values such as sustainability or quality.
- Meet the Team Section:
 - Photos and short bios of the founders or team members.
- Special Features Section:
 - *"Why Choose Us?"*
 - Highlight USP (organic beans, specialty brews, etc.).
- Footer: (same as home page).

3. Products Page

Purpose:

- Showcase the menu and services offered.

Content Plan:

- Header: (same as home page).
- Menu Sections:
 - Category: e.g., Coffee, Tea, Snacks, Desserts.
 - Product tiles: Images, names, short descriptions, prices.
- Highlight Section:
 - *"Top Picks"* or *"Customer Favorites."*
- CTA Section:
 - Button: *"Order Now"* linking to Login or Registration page.
- Footer: (same as home page).

4. Testimonials Page

Purpose:

- Build trust by showcasing feedback from happy customers.

Content Plan:

- Header: (same as home page).
- Customer Feedback Section:
 - Quotes or testimonials from existing customers.
 - Option to display Google or Yelp reviews.
 - Use star ratings for visual appeal.
- Submit a Testimonial:
 - Simple form to allow visitors to submit reviews.
- Footer: (same as home page).

5. Contact Page

Purpose:

- Allow customers to reach out easily for inquiries, reservations, or feedback.

Content Plan:

- Header: (same as home page).
- Contact Form:
 - Name, Email, Subject, Message.
 - Submit button with form validation.
- Location Section:
 - Embedded Google Map showing the Fragrance shop's location.
- Operating Hours Section:
 - Business hours listed clearly.
- Footer: (same as home page).

6. Login Page

Purpose:

- Enable existing users to log in to their accounts.

Content Plan:

- Form:
 - Email and Password fields.
 - Submit button.
- Forgot Password Link:
 - Redirects to password recovery.
- CTA:
 - Link to the Registration page: *"Don't have an account? Sign Up Now!"*.

7. Registration Page

Purpose:

- Allow new users to register for an account.

Content Plan:

- Form Fields:
 - Full Name.
 - Email.
 - Password and Confirm Password.
- Form Validation:
 - Password requirements.
- Submit Button:
 - Validates data and submits.
- Footer: (same as home page).

9. Add ideas for content, images & layout

Below are detailed ideas for crafting the content, selecting images, and structuring the layout for a Fragrance shop website with at least five pages and login/registration functionality.

1. Home Page

Layout Ideas:

- Header Section:
 - Fixed navigation bar.
 - Logo on the left, menu items in the center, and a Login/Register button on the right.
- Hero Section:
 - Full-width background image of the Fragrance shop or coffee art.
 - Overlay text with a tagline (e.g., "Brewed with Love!").
 - A button like "Explore Menu" or "Visit Us."
- Featured Section:
 - Use cards or tiles to display 3-4 featured items (e.g., "Latte of the Day" or seasonal beverages).
 - Include enticing images and brief descriptions.

Content Ideas:

- Short, catchy welcome message.
- Announcements for specials or promotions, like "Happy Hour: 3 PM to 5 PM!".
- Direct link to the menu page.

Image Ideas:

- A banner of freshly brewed coffee.
- Close-up shots of coffee beans, latte art, and cozy seating areas.

2. About Page

Layout Ideas:

- Story Section:
 - A timeline or column layout telling the story of the Fragrance shop.
 - Use dividers or icons to separate milestones.
- Team Section:
 - Grid layout featuring team member images and short bios.

- Feature Section:
 - Icons and text showcasing special features (e.g., Organic beans, Sustainable practices, etc.).

Content Ideas:

- Mission statement, history, and values.
- Insights about sourcing and partnerships.
- Emphasis on community connection.

Image Ideas:

- Images of founders, the coffee-making process, and snapshots of community events.
- Collage of the shop's interior.

3. Products/Services Page

Layout Ideas:

- Categories Section:
 - Split the page into categories like *Coffee*, *Pastries*, *Teas*, and *Special Offers*.
 - Each category features product images, names, descriptions, and prices.
- Highlight Section:
 - Slider showing customer favorites or best sellers.
- CTA Section:
 - *Order Now* or *Learn More* buttons under each item.

Content Ideas:

- Detailed descriptions of items, focusing on unique flavors or preparation techniques.
- Highlight seasonal specialties (e.g., "Pumpkin Spice Latte").

Image Ideas:

- High-resolution images of coffee cups, pastries, tea varieties.
- Display trays, artisan-style packaging.

4. Testimonials/Reviews Page

Layout Ideas:

- Feedback Section:
 - Use a carousel to rotate customer reviews.
 - Display text alongside star ratings.
- CTA Section:
 - A "Submit Your Testimonial" form on the bottom.

Content Ideas:

- Customer feedback emphasizing ambiance, taste, and service.
- Pull quotes from external reviews (Google, Yelp, etc.).

Image Ideas:

- Happy customers enjoying coffee.
- Stylized images of written reviews.

5. Contact Page

Layout Ideas:

- Form Section:
 - A simple contact form (fields for name, email, subject, message).
- Map Section:
 - Embed Google Maps for easy navigation.
- Info Section:
 - Use icons to display the address, phone number, and operating hours.

Content Ideas:

- A friendly *"We'd love to hear from you!"* message.
- FAQs for common inquiries.

Image Ideas:

- Fragrance shop storefront.
- Icons or illustrations for contact methods.

6. Login Page

Layout Ideas:

- Minimalist form interface.
- A side panel or banner featuring the shop logo or relevant imagery.

Content Ideas:

- Fields for email and password.
- A friendly reminder like: *"New here? Sign up now!"*

Image Ideas:

- Illustration of a coffee cup or beans.
- Background image with a transparent overlay for text fields.

7. Registration Page

Layout Ideas:

- Clean, centered form fields.
- Password strength indicator.

Content Ideas:

- A tagline: *"Join us and enjoy exclusive rewards!"*.
- Checkbox for opting into newsletters or special offers.

Image Ideas:

- A welcoming banner image.
- Icons for form fields (e.g., lock for password).

8. Visual Design Ideas for All Pages:

Colors:

- Earth tones: brown, cream, and green.
- Highlight color: gold or maroon for buttons and accents.

Fonts & Typography:

- Use a warm, inviting font for headers (e.g., *Playfair Display*).
- Clean sans-serif font for body text (e.g., *Lato* or *Roboto*).

Logos:

- Stylized coffee cup with steam.

Images:

- Realistic, high-quality visuals with natural lighting.
- Use

10. Determine your site structure or Create content for your core website pages:

- Home page**
- About page**
- Product/Service page**
- Contact page**
- Starter blog posts**

1. Home Page

The home page serves as the introduction to the Fragrance shop and provides navigation to the key sections of the website.

Sections and Content:

- Header:
 - Logo: A simple, bold representation of the Fragrance shop (e.g., coffee cup, beans).
 - Navigation Links: Menu, About, Products, Testimonials, Contact.
 - Call-to-Action Button: *"Order Now"* or *"Join Us"* (links to Order page or Registration page).
- Hero Section:
 - Full-width background image of Fragrance shop or popular drinks.
 - Text Overlay: *"Brewing the Perfect Cup, One Sip at a Time!"*
 - A call-to-action button like *"Explore Our Menu"* or *"Visit Us Today"*.
- About Section (Teaser):
 - Short paragraph introducing the Fragrance shop, inviting visitors to learn more about its story.
 - Link to the *About* page.
- Product Highlights Section:
 - Grid of 3-4 featured products: *"Coffee of the Week," "Specialty Latte," "Pastries."*
 - Images and short descriptions with an option to learn more or place an order.
- Social Proof Section (Testimonial Teaser):
 - Snippets from customer reviews with a *"See More"* button linking to the full testimonials page.
- Footer:
 - Quick links (menu, shop hours, locations, FAQs).
 - Social media icons (Facebook, Instagram, Twitter).
 - Store Address and a Google Map embed.

2. About Page

The about page provides background on the Fragrance shop, its mission, values, and team.

Sections and Content:

- Introduction:
 - Overview of the Fragrance shop's story, mission, and core values.
 - Brief history of the business: "Founded in 2020 with a passion for community and great coffee..."
- Meet the Team:
 - Grid layout for team members with their names, photos, and brief bios. Focus on baristas, managers, and key staff.
- Our Promise:
 - Information about coffee sourcing, sustainability, and quality control.
 - Icons or short bullet points explaining eco-friendly practices like compostable cups, local sourcing, and more.
- Location Section:
 - Locations of stores with Google map integration.
 - Hours of operation.

3. Product/Service Page

This page showcases the Fragrance shop's offerings with a focus on the products, their descriptions, and an easy way for visitors to place an order.

Sections and Content:

- Coffee and Drink Section:
 - Show different categories of drinks: Hot Coffee, Iced Drinks, Specialty Lattes, etc.
 - Each product is shown with an image, short description (flavors, ingredients), and price.
 - A "Add to Cart" button or "Order Now" linking to the order system.
- Food Section:
 - Showcase food items: Cakes, Muffins, Sandwiches, etc., with images and descriptions.
 - Prices included.
- Popular Items & Limited-Time Specials:
 - Carousel or featured box for seasonal items and the "Drink of the Month".
- Order Online:
 - Options to choose from a mobile ordering platform or an in-house system that integrates with the website. Include delivery and pick-up details.
- Footer (same as Home Page):
 - Quick links, social media icons, store locations, etc.

4. Contact Page

The contact page allows customers to reach out to the Fragrance shop directly and learn more about visiting.

Sections and Content:

- Contact Form:
 - Name, email, message fields for direct inquiries.
 - "Submit" button.
- Social Media & Address Section:

- Social media icons that link to the Fragrance shop's Facebook, Instagram, Twitter.
- A full street address with phone number, email, and operating hours.
- Interactive Map:
 - Google Maps integration for directions to the physical Fragrance shop location.
- Support Information:
 - Section to contact customer support, along with a few FAQs.

5. Login & Registration Pages

These pages allow users to create accounts for order tracking, loyalty programs, and to receive personalized updates.

- Login Page:
 - User credentials fields: Username/email, password.
 - Button to submit login details.
 - "Forgot password?" link.
- Registration Page:
 - Fields to sign up: Name, email, password.
 - Option to subscribe to a newsletter or loyalty program.

6. Overall Website Structure Map:

- Home Page: Introduction and links to sections, including featured products, reviews, and social media.
- About Page: History, team members, and values of the Fragrance shop.
- Product/Service Page: Showcases offerings with pricing and the order option.
- Testimonials/Review Page: User feedback section with submission form.
- Contact Page: Contact form, store location, and links to social media.
- Blog Section: Knowledge-sharing blog posts for customer engagement.
- Login/Registration Page: For order tracking and special offers.

11. Create and collect design elements

These design elements define your brand personality and help customers feel what your brand represents through the use of:

- a. Colors
- b. Fonts and typography
- c. Logos
- d. Images and photos

Design Elements Defining Brand Personality for Fragrance shop Website:

The design elements of a website are vital in creating an identity and establishing a strong emotional connection between the brand and its visitors. For your Fragrance shop website, the goal is to create a space that feels inviting, cozy, and professional, much like the experience customers would have in the Fragrance shop itself. Here's a breakdown of how the design elements for your website can define and enhance your brand's personality.

1. Colors

Colors have the power to influence mood and perceptions. The color palette should reflect your Fragrance shop's cozy, welcoming atmosphere, as well as its values (such as quality, sustainability, and warmth).

- Primary Colors:
 - Coffee Brown (#6F4F37): This warm, earthy brown can symbolize the coffee itself. It evokes feelings of comfort, sophistication, and richness. Perfect for backgrounds, headers, and navigation bars.
 - Cappuccino Beige (#D8CAB8): This soft, neutral color is perfect for backgrounds, light section dividers, or even for highlights. It's comforting and easy on the eyes, lending a calming and welcoming vibe.
 - Cream (#F1E0C6): A light, soothing color that can be used for secondary areas to maintain clarity and balance. It pairs well with darker tones and adds a light, airy feel.
- Accent Colors:
 - Espresso Black (#2B1B1D): A deep, sleek color to be used for text, CTA (call-to-action) buttons, or footer sections. It maintains readability while establishing a professional tone.
 - Rich Green (#6DBF3A): A shade reflecting sustainability and organic values. Use this for CTAs (like *"Order Now"* or *"Join Us"*), navigation items, or promotional sections.

Psychological Impact: These colors communicate warmth, comfort, and natural, high-quality ingredients—making it a space people want to return to. Green accents will also create a fresh, eco-friendly atmosphere.

2. Fonts and Typography

The typography should convey a professional yet cozy feel, matching your brand's personality.

- Heading Font:
 - Playfair Display (serif) or Lora: These elegant fonts have an old-fashioned charm, which works well for headings and subheadings on the homepage and menu pages. It represents traditional coffee culture with a modern twist.
- Body Font:
 - Open Sans or Roboto (sans-serif): Clean, modern, and highly readable. The body text needs to be easy on the eyes since customers will spend time reading product descriptions or information about the Fragrance shop. This font should be used for paragraphs, blogs, and menu text.
- Font Weights:
 - Use bold or semi-bold weight for headings to create visual hierarchy, and regular font weights for text to ensure ease of reading.

Impact: The mix of serif and sans-serif fonts maintains a balance between tradition and modernity, perfect for a Fragrance shop with a warm, upscale yet modern experience.

3. Logo

Your logo represents the visual identity of your Fragrance shop and sets the tone for your brand's story.

- Logo Design: The logo should be simple but memorable, combining visual elements that represent coffee. Consider using stylized coffee cups, coffee beans, or steam swirls. These visuals should clearly associate the logo with the essence of the shop.

- Color Palette for the Logo: Use the primary colors like coffee brown and cappuccino beige, along with a touch of espresso black for contrast. If your Fragrance shop values organic ingredients, incorporating a bit of green could reinforce the sustainability aspect.

Logo Usage:

- The logo should be placed prominently at the top of each page in the header.
- Ensure its scalability for use on print material, social media, packaging, and within the header of your website.

Impact: The logo serves as the face of your brand, instantly giving customers a sense of the quality and warmth they can expect when visiting your physical or digital space.

4. Imagery and Photos

Imagery on the Fragrance shop website has the ability to build a stronger emotional connection by showcasing the coffee experience.

- Product Photography: High-quality images of coffee drinks, pastries, and desserts should dominate the site. Think close-ups of frothy cappuccinos, beans being ground, steaming cups of coffee, or beautiful latte art.
 - For the menu page, show clean, professional shots of the products with descriptions.
 - For the about page, images of the interior of your Fragrance shop, people enjoying their coffee, or team photos add authenticity and a sense of community.
- Ambiance Photography: Show the cozy Fragrance shop setting with soft lighting, wooden tables, and greenery. These images should showcase the atmosphere visitors will experience in person. Consider using candid photos of customers enjoying coffee together or a barista preparing a drink.
- Lifestyle Photography: In addition to product-specific images, showing a lifestyle—people studying, working, or socializing in your café can be powerful. This reinforces the idea that your shop is a place to gather, relax, and socialize.

Impact: High-quality, authentic photos will create a warm, welcoming ambiance, making the site feel as inviting as the physical shop itself. They offer a visual sense of what it feels like to enjoy a cup of coffee in your space.

5. Interactive Elements and Buttons

To make sure that the design is functional, interactive elements must be seamlessly integrated, improving the overall experience while keeping in line with your brand's identity.

- Navigation Buttons: Ensure that buttons like “Order Now,” “Book a Table,” and “Join Our Newsletter” are easy to see. Use accent colors like rich green or espresso black to make CTAs stand out without being overwhelming.
 - Use hover effects to indicate interactiveness (such as a light shadow or background color change).
- Icons: To enhance the user experience, icons should represent different site sections (like a coffee cup for the menu, a pin for locations, or a heart for the shopping cart). Simple, clean icons that match your brand colors will guide the customer through the site intuitively.
 - Use lightbulb iconography for new ideas or specials.

Conclusion:

The Fragrance shop Website project serves as a comprehensive exercise in understanding and applying web development fundamentals. Through careful planning and implementation of design elements, content organization, and user experience considerations, the assignment ensures a functional, visually appealing, and user-friendly platform.

By creating essential pages like the Home, About, Product/Service, Testimonial, Contact, and Login/Registration pages, the website provides customers with intuitive navigation and engaging content. The visual direction, including the use of harmonious colors, typography, logos, and images, helps reinforce the brand's identity and appeal to the target audience.

This assignment highlights the importance of a structured approach to web design—starting with setting goals and mapping out the project structure, followed by planning and designing the content. By addressing audience needs and incorporating user-centric solutions, the project enhances the online presence of a Fragrance shop while delivering a seamless digital experience for customers.

Experiment No.2

Problem Statement:

HTML

- Create a detailed home page for the Fragrance shop website.

- Create a detailed menu/product page for the Fragrance shop website, listing all available items categorized appropriately.
- Create a cart page that allows customers to review and manage the items they wish to purchase before proceeding to checkout.
- Create an about us page that provides detailed information about the Fragrance shop's history, mission, and team.
- Create a contact page that allows customers to easily get in touch with the Fragrance shop through a form.
- Design and implement admin/user registration form for the Fragrance shop website.
- Design and implement admin/user login form for the Fragrance shop website

Objective:

To construct the foundational semantic HTML structure for the core pages of the fragrance shop e-commerce platform, including product displays, cart management, and user authentication forms.

Theory:

Deep Theoretical Background: HTML (HyperText Markup Language) is a structural markup language. When a browser parses an HTML document, the rendering engine constructs the **Document Object Model (DOM)**—a programmatic, tree-like structure where every HTML tag becomes a node (an object).

1. The Semantic Web and Accessibility (a11y): In early web development, pages were constructed using generic `<div>` (division) tags, often called "div soup." HTML5 introduced **Semantic Markup**. Tags like `<header>`, `<nav>`, `<main>`, `<section>`, and `<footer>` carry inherent structural meaning.

- **Accessibility:** Screen readers used by visually impaired users rely on the DOM tree to navigate. A semantic `<nav>` tag tells the software "this is a navigation menu," allowing the user to skip it. Generic `<div>` tags provide no such context.
- **Search Engine Optimization (SEO):** Web crawlers (like Googlebot) use semantic tags to determine the importance of content. Text inside an `<h1>` or `<main>` tag is weighted heavier than text in a `<aside>`.

2. Form Architecture and HTTP Protocols: The `<form>` element is the primary gateway for user-to-server data transmission.

- **Method Attribute:** Forms use the HTTP protocol to communicate. `method="GET"` appends form data to the URL (useful for search bars but insecure for passwords). `method="POST"` encapsulates the data within the body of the HTTP request, which is mandatory for transmitting sensitive authentication data securely.
- **Input Types:** Using specific input types (`type="email"`, `type="password"`) triggers specialized browser behaviors. For instance, `type="password"` masks the characters for "shoulder-surfing" security, and `type="email"` triggers the @ symbol on mobile device keyboards.

Code:

A. Home page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Home</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>AURA Luxury Fragrances</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="products.html">Shop</a> |
      <a href="about.html">About Us</a> |
      <a href="contact.html">Contact</a> |
      <a href="cart.html">Cart</a> |
      <a href="login.html">Login</a>
    </nav>
  </header>
  <hr>

  <main class="home-container">
    <section class="hero-banner">
      <h2>Discover Your Signature Scent</h2>
      <p>Explore our exclusive collection of luxury perfumes, masterfully crafted with the rarest botanical ingredients from around the world.</p>
      <br>
      <a href="products.html"><button class="btn-primary">Explore the Collection</button></a>
    </section>

    <section class="featured-categories" style="text-align: center; margin-top: 40px;">
      <h2>Curated Collections</h2>
      <p>Find the perfect olfactory expression for every occasion.</p>
      <br>
      <div style="display: flex; justify-content: space-around; flex-wrap: wrap;">
        <div>
          <h3>Pour Femme</h3>
          <p>Elegant, floral, and deeply romantic silhouettes.</p>
        </div>
        <div>
          <h3>Pour Homme</h3>
          <p>Bold, woody, and sophisticated classics.</p>
        </div>
        <div>
          <h3>L'Essence (Unisex)</h3>
          <p>Avant-garde blends that transcend traditional boundaries.</p>
        </div>
      </div>
    </section>
  </main>

  <hr>
  <footer>
    <p>&copy; 2026 Aura Fragrances. All rights reserved.</p>
```

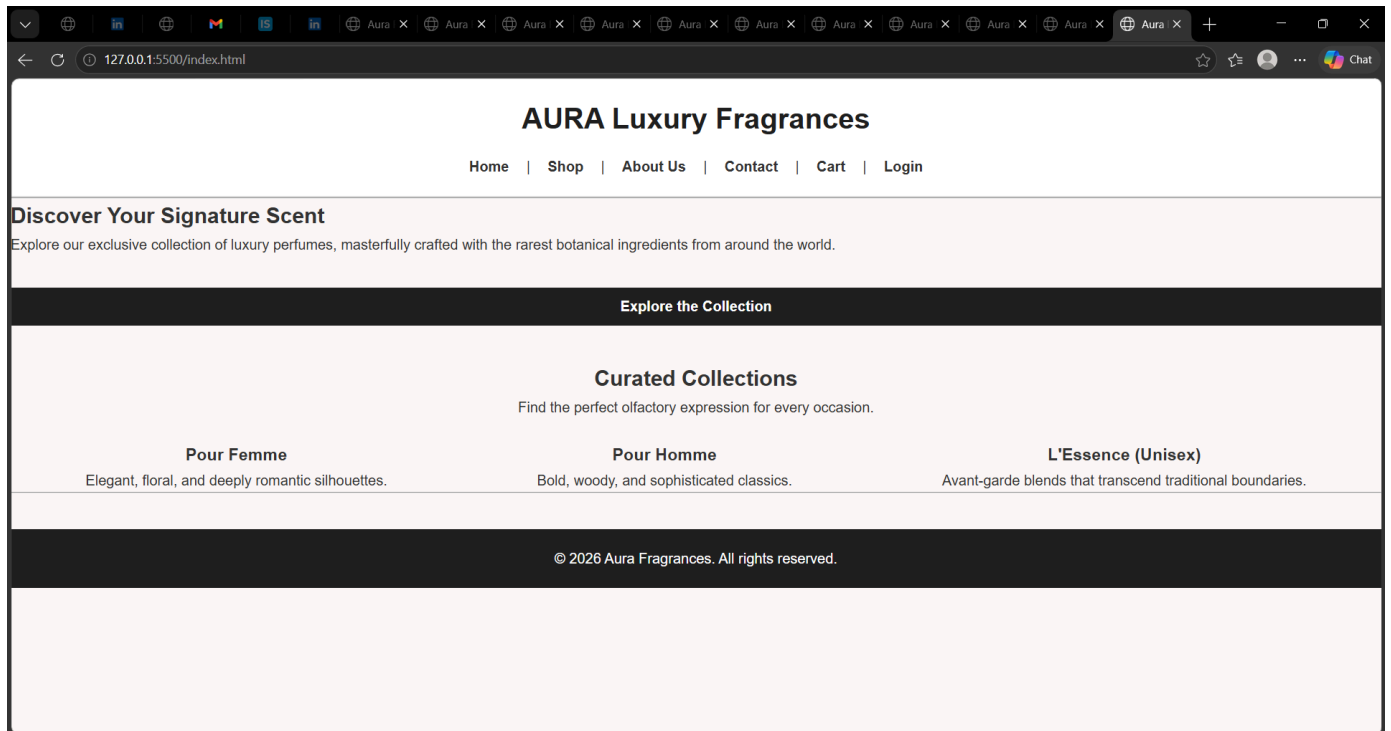
```

</footer>
</body>
</html>

```

Output:

A. Index/Home page output:



Code:

B. menu/product page:

code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Shop</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Our Fragrance Collection</h1>
    <nav>
      <a href="index.html">Back to Home</a> |
      <a href="cart.html">View Cart</a>
    </nav>
  </header>
  <hr>

  <main>
    <section class="category-section">
      <h2 class="category-heading">Women's Collection</h2>

```

```

<hr class="category-separator">

<div class="product-grid">
  <div class="product-card">
    
    <h3 class="product-title">Midnight Rose</h3>
    <p class="product-description"><strong>Top:</strong> Bergamot <br><strong>Heart:</strong>
Damask Rose <br><strong>Base:</strong> Patchouli, Amber</p>
    <p class="product-price">$125.00</p>
    <button class="btn-primary" onclick="addToCart('Midnight Rose', 125.00)">Add to Cart</button>
  </div>

  <div class="product-card">
    
    <h3 class="product-title">Vanilla Silk</h3>
    <p class="product-description"><strong>Top:</strong> Almond <br><strong>Heart:</strong>
Jasmine <br><strong>Base:</strong> Madagascar Vanilla</p>
    <p class="product-price">$110.00</p>
    <button class="btn-primary" onclick="addToCart('Vanilla Silk', 110.00)">Add to Cart</button>
  </div>
</div>
</section>

<section class="category-section">
  <h2 class="category-heading">Men's Collection</h2>
  <hr class="category-separator">

  <div class="product-grid">
    <div class="product-card">
      
      <h3 class="product-title">Oceanic Wood</h3>
      <p class="product-description"><strong>Top:</strong> Sea Salt, Grapefruit
<br><strong>Heart:</strong> Driftwood <br><strong>Base:</strong> Cedar</p>
      <p class="product-price">$95.00</p>
      <button class="btn-primary" onclick="addToCart('Oceanic Wood', 95.00)">Add to Cart</button>
    </div>

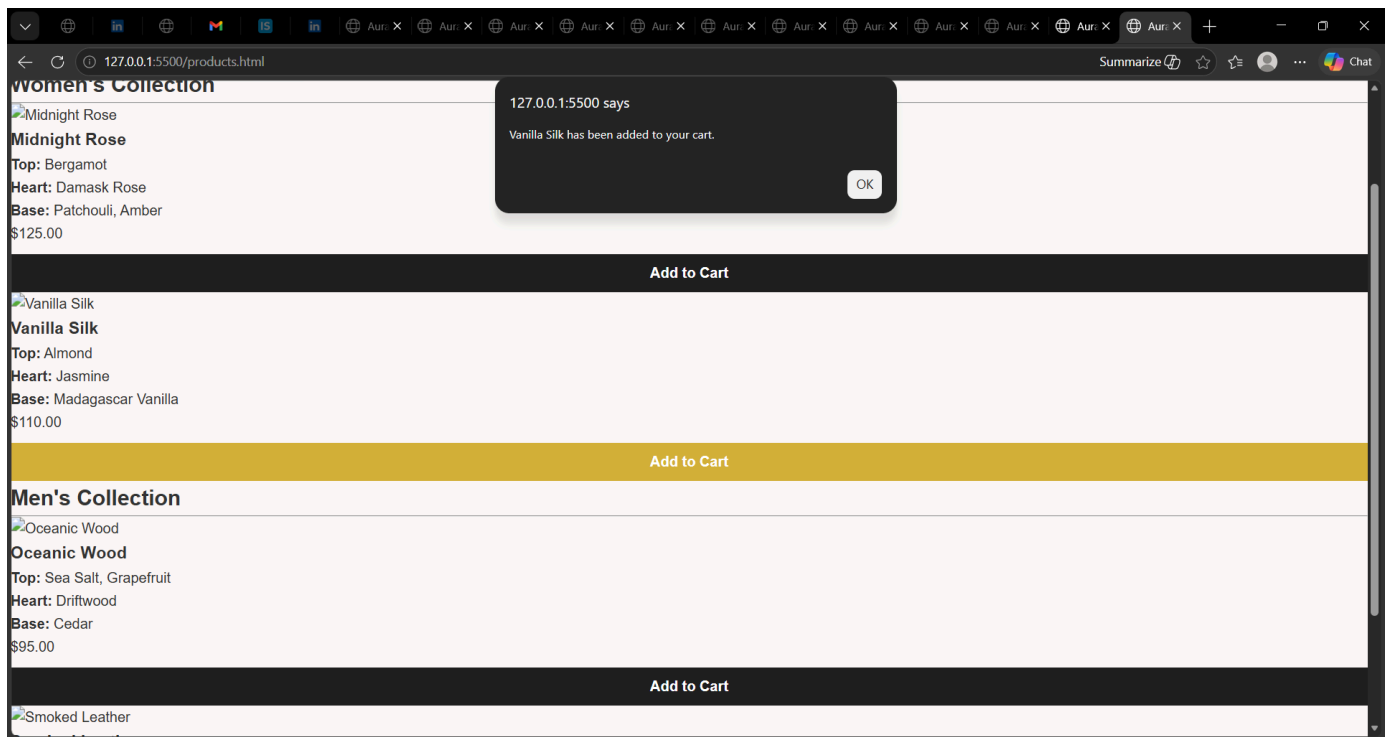
    <div class="product-card">
      
      <h3 class="product-title">Smoked Leather</h3>
      <p class="product-description"><strong>Top:</strong> Black Pepper
<br><strong>Heart:</strong> Tobacco Leaf <br><strong>Base:</strong> Rich Leather</p>
      <p class="product-price">$130.00</p>
      <button class="btn-primary" onclick="addToCart('Smoked Leather', 130.00)">Add to
Cart</button>
    </div>
  </div>
</section>
</main>

<script src="script.js"></script>
</body>
</html>

```

Output:

B. menu/product page output:



Code:

C. cart page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Shopping Cart</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Your Shopping Cart</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="products.html">Continue Shopping</a>
    </nav>
  </header>
  <hr>

  <main class="cart-container">
    <table class="cart-table">
      <thead>
        <tr>
          <th>Product Details</th>
          <th>Price</th>
          <th>Quantity</th>
          <th>Total</th>
          <th>Action</th>
        </tr>
      </thead>
```

```

<tbody>
  <tr>
    <td><strong>Midnight Rose</strong><br><span style="font-size: 0.8rem; color: #666;">Size: 50ml
Eau de Parfum</span></td>
    <td>$125.00</td>
    <td><input type="number" value="1" min="1" style="width: 50px; padding: 5px;"></td>
    <td>$125.00</td>
    <td><button class="btn-remove">Remove</button></td>
  </tr>
</tbody>
<tfoot>
  <tr>
    <td colspan="3"
align="right"><strong>Subtotal:</strong><br><strong>Shipping:</strong><br><strong>Grand
Total:</strong></td>
    <td colspan="2">
      $125.00<br>Free<br>
      <strong style="color: #d4af37; font-size: 1.5rem;">$125.00</strong>
    </td>
  </tr>
</tfoot>
</table>

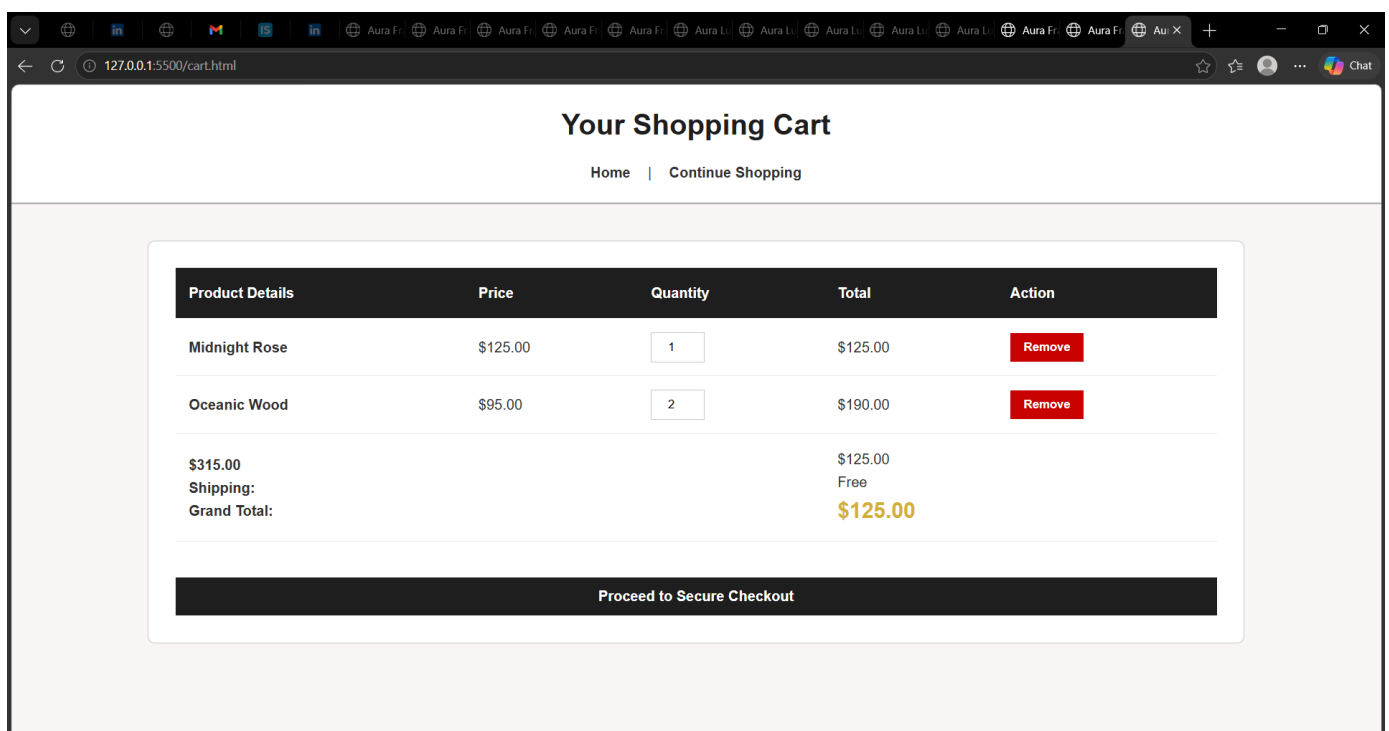
<div class="cart-actions" style="text-align: right; margin-top: 20px;">
  <a href="checkout.php"><button class="btn-primary">Proceed to Secure Checkout</button></a>
</div>
</main>

<script src="script.js"></script>
</body>
</html>

```

Output:

C. cart page output:



Code:

D. about us page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | About Us</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>The Story of Aura</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="products.html">Shop</a>
    </nav>
  </header>
  <hr>

  <main class="about-container">
    <h2>Our Heritage</h2>
    <p>Founded by master perfumers in 2026, Aura Fragrances was established with a singular, passionate vision: to restore true artistry, emotion, and craftsmanship to modern perfumery. We believe that a scent is an invisible, deeply personal signature that leaves a lasting impression long after you have left the room.</p>

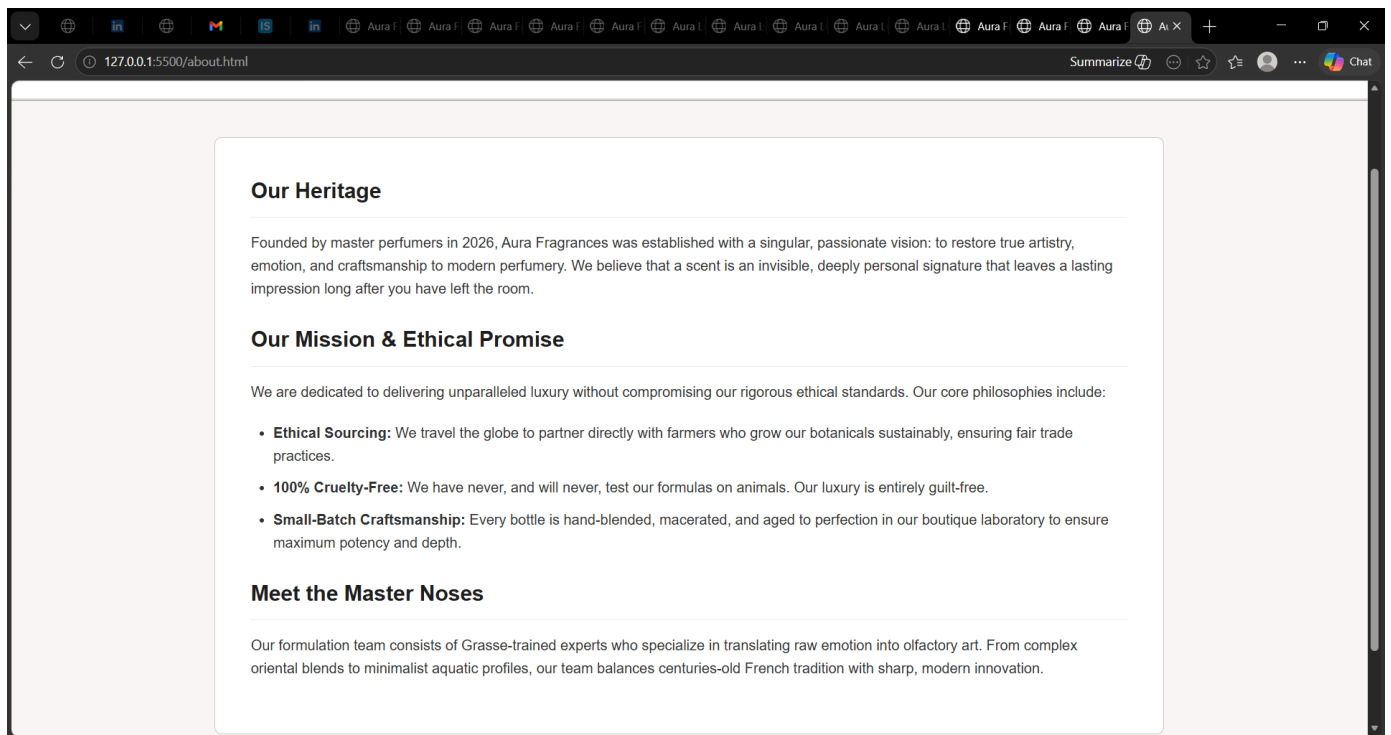
    <h2>Our Mission & Ethical Promise</h2>
    <p>We are dedicated to delivering unparalleled luxury without compromising our rigorous ethical standards. Our core philosophies include:</p>
    <ul>
      <li><strong>Ethical Sourcing:</strong> We travel the globe to partner directly with farmers who grow our botanicals sustainably, ensuring fair trade practices.</li>
      <li><strong>100% Cruelty-Free:</strong> We have never, and will never, test our formulas on animals. Our luxury is entirely guilt-free.</li>
      <li><strong>Small-Batch Craftsmanship:</strong> Every bottle is hand-blended, macerated, and aged to perfection in our boutique laboratory to ensure maximum potency and depth.</li>
    </ul>

    <h2>Meet the Master Noses</h2>
    <p>Our formulation team consists of Grasse-trained experts who specialize in translating raw emotion into olfactory art. From complex oriental blends to minimalist aquatic profiles, our team balances centuries-old French tradition with sharp, modern innovation.</p>
  </main>

  <footer>
    <p>&copy; 2026 Aura Fragrances. All rights reserved.</p>
  </footer>
</body>
</html>
```

Output:

D. about us page output:



Code:

E. contact us page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Contact</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Get in Touch</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="about.html">About Us</a>
    </nav>
  </header>
  <hr>

  <main>
    <div class="form-container">
      <h2>Send us a Message</h2>
      <form action="#" method="POST">
        <div class="form-group">
          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name" placeholder="Enter your full name" required>
        </div>

        <div class="form-group">
          <label for="email">Email Address:</label>
```



```

        <input type="email" id="email" name="email" placeholder="Enter your email" required>
    </div>

    <div class="form-group">
        <label for="subject">Inquiry Type:</label>
        <select id="subject" name="subject" style="width: 100%; padding: 12px; margin-bottom: 1.5rem;
border: 1px solid #ccc; border-radius: 4px;">
            <option value="scent_advice">Scent Recommendation</option>
            <option value="order_issue">Order Status</option>
            <option value="general">General Inquiry</option>
        </select>
    </div>

    <div class="form-group">
        <label for="message">Your Message:</label>
        <textarea id="message" name="message" rows="5" placeholder="How can our fragrance experts
assist you?" required></textarea>
    </div>

    <button type="submit" class="btn-primary">Submit Inquiry</button>
</form>
</div>
</main>
</body>
</html>

```

Output:

E. contact us page output:

The screenshot displays a web browser window with the address bar showing '127.0.0.1:5500/contact.html'. The page has a navigation bar with 'Home' and 'About Us' links. The main content area features a form titled 'Send us a Message'. The form contains the following elements:

- Full Name:** A text input field with the placeholder text 'Enter your full name'.
- Email Address:** A text input field with the placeholder text 'Enter your email'.
- Inquiry Type:** A dropdown menu with 'Scent Recommendation' selected.
- Your Message:** A text area with the placeholder text 'How can our fragrance experts assist you?'.

Code:

F. registration page:

code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Register</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Join the Aura Society</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="login.html">Login</a>
    </nav>
  </header>
  <hr>

  <main>
    <div class="form-container">
      <h2>Create Your Account</h2>
      <p style="text-align: center; margin-bottom: 20px; color: #666;">Register to track orders and receive
exclusive access to private launches.</p>

      <form id="register-form" action="register.php" method="POST">
        <div class="form-group">
          <label for="fname">Full Name:</label>
          <input type="text" id="fname" name="fname" placeholder="Enter your full name" required>
        </div>

        <div class="form-group">
          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" placeholder="name@example.com" required>
        </div>

        <div class="form-group">
          <label for="password">Create Password:</label>
          <input type="password" id="password" name="password" placeholder="Minimum 8 characters"
required>
        </div>

        <div class="form-group">
          <label for="confirm_pass">Confirm Password:</label>
          <input type="password" id="confirm_pass" name="confirm_pass" placeholder="Re-enter your
password" required>
        </div>

        <button type="submit" class="btn-primary">Register Account</button>
      </form>

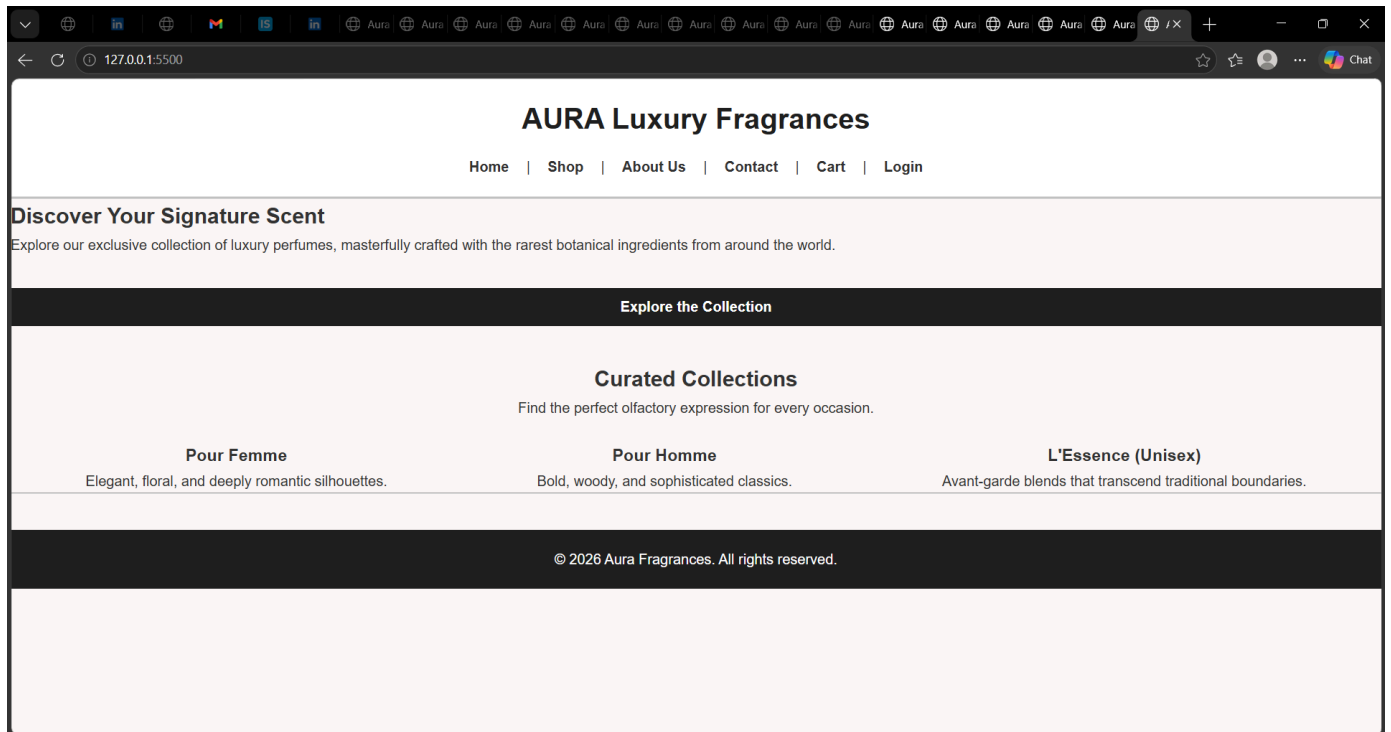
      <p style="text-align: center; margin-top: 20px; font-size: 0.9rem;">
        Already a society member? <a href="login.html" style="color: #d4af37; font-weight: bold;
text-decoration: none;">Log in here</a>.
      </p>
    </div>
  </main>

```

```
<script src="script.js"></script>
</body>
</html>
```

Output:

F. registration page output:



Code:

G. login page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Login</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Member Login</h1>
    <nav>
      <a href="index.html">Home</a> |
      <a href="register.html">Register</a>
    </nav>
  </header>
  <hr>

  <main>
    <div class="form-container">
      <h2>Welcome Back</h2>
```

```

<p style="text-align: center; margin-bottom: 20px; color: #666;">Please sign in to access your
account.</p>

<form id="login-form" action="login.php" method="POST">
  <div class="form-group">
    <label for="login_email">Email Address:</label>
    <input type="email" id="login_email" name="email" placeholder="Enter your registered email"
required>
  </div>

  <div class="form-group">
    <label for="login_password">Password:</label>
    <input type="password" id="login_password" name="password" placeholder="Enter your
password" required>
  </div>

  <button type="submit" class="btn-primary">Sign In</button>
</form>

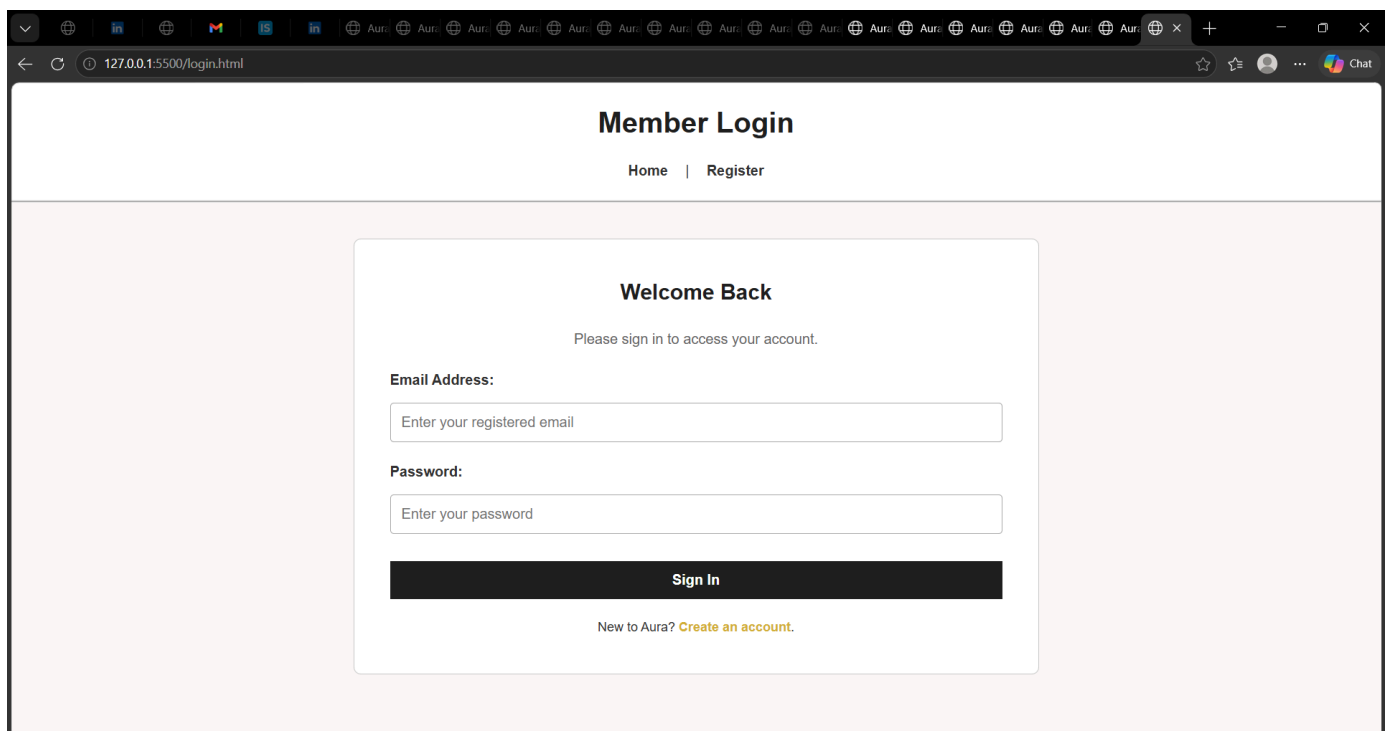
<p style="text-align: center; margin-top: 20px; font-size: 0.9rem;">
  New to Aura? <a href="register.html" style="color: #d4af37; font-weight: bold; text-decoration:
none;">Create an account</a>.
</p>
</div>
</main>

<script src="script.js"></script>
</body>
</html>

```

Output:

G. login page output:



Conclusion: This experiment successfully demonstrated the implementation of a robust, accessible Document Object Model (DOM) using HTML5 semantic markup. By utilizing tags such as `<header>`, `<main>`, `<section>`, and `<nav>` rather than generic `<div>` containers, the web application was optimized for both Search Engine Optimization (SEO) and web accessibility (screen readers). Furthermore, the precise construction of data-collection endpoints using HTML `<form>` elements and specialized input types (`email`, `password`) established the necessary client-side infrastructure to securely transmit user authentication and contact data to a backend server.

Experiment No.3

Problem Statement:

- Enhance the layout of the Fragrance shop website using CSS Grid for the home page.
- Use CSS Grid to layout the menu/product items in a structured and style the menu categories with appropriate headings, spacing, separators, images, descriptions, and prices.

Objective:

To implement CSS Grid architecture to create responsive and visually structured layouts for the fragrance shop's homepage and product catalog.

Theory:

Deep Theoretical Background: When the browser combines the DOM with the **CSSOM (CSS Object Model)**, it creates the Render Tree. The layout phase calculates the exact geometry of every node. The **CSS Grid Layout Module** is a sophisticated, two-dimensional layout algorithm that fundamentally changes how this geometry is calculated.

1. Two-Dimensional Spatial Distribution: Historically, web layout was constrained to one dimension at a time (using Floats or Flexbox). CSS Grid allows developers to define a grid formatting context for both rows and columns simultaneously.

- **Grid Terminology:** A layout consists of the *Grid Container* (the parent), *Grid Lines* (the invisible dividing lines), *Grid Tracks* (the space between two lines, i.e., a row or column), and *Grid Cells* (the intersection of a row and column).

2. Intrinsic Sizing and the Fractional Unit (fr): Grid introduces the **fr** (fractional) unit. If a container has `grid-template-columns: 1fr 2fr;`, the browser calculates the total available width, divides it by 3, gives one part to the first column, and two parts to the second. This mathematical fluidly replaces static pixel layouts.

3. Algorithmic Responsiveness: The hallmark of modern CSS Grid is responsive autoplacement. By combining the `repeat()` function with `auto-fit` and `minmax(250px, 1fr)`, the CSS engine executes a dynamic algorithm: "Create as many columns as possible that are at least 250 pixels wide. If the viewport shrinks and a 250px column cannot fit, automatically push the item to the next row and distribute the remaining space equally." This creates a fluid product catalog without relying on complex, device-specific CSS Media Queries.

Code:

A. styles.css:

code:

```
* {
  margin: 0;
  padding: 0;
  box-sizing: border-box;
}

body {
  font-family: 'Helvetica Neue', Arial, sans-serif;
  background-color: #faf9f6; /* Luxury off-white background */
  color: #2c2c2c;
  line-height: 1.6;
}

h1, h2, h3 {
  font-family: 'Georgia', serif;
  color: #1a1a1a;
}

header {
  text-align: center;
  padding: 2rem;
  background-color: #ffffff;
  border-bottom: 2px solid #d4af37; /* Gold accent line */
}

header h1 {
  font-size: 2.5rem;
  letter-spacing: 2px;
  margin-bottom: 1rem;
}

nav a {
  text-decoration: none;
```

```

    color: #666666;
    margin: 0 15px;
    font-weight: bold;
    text-transform: uppercase;
    font-size: 0.9rem;
    transition: color 0.3s ease;
}

nav a:hover {
    color: #d4af37;
}

/* Base Horizontal Rule */
hr {
    border: 0;
    height: 1px;
    background-color: #e0e0e0;
    margin: 20px 0;
}

button, .btn-primary {
    background-color: #2c2c2c;
    color: #ffffff;
    border: none;
    padding: 12px 20px;
    cursor: pointer;
    text-transform: uppercase;
    font-weight: bold;
    border-radius: 4px;
    transition: background-color 0.3s ease;
    width: 100%;
}

button:hover, .btn-primary:hover {
    background-color: #d4af37;
}

.home-container {
    display: grid;
    grid-template-columns: 1fr;
    gap: 40px;
    padding: 40px 10%;
}

.hero-banner {
    background-color: #2c2c2c;
    color: #ffffff;
    text-align: center;
    padding: 60px 20px;
    border-radius: 8px;
}

.hero-banner h2 {
    font-size: 2.5rem;
    margin-bottom: 15px;
    color: #d4af37;
}

```

```

/* Using CSS Grid for the 3 Featured Categories on the Home Page */
.featured-categories > div {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(250px, 1fr));
  gap: 20px;
  margin-top: 20px;
}

.featured-categories div > div {
  background-color: #ffffff;
  border: 1px solid #e0e0e0;
  padding: 30px 20px;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.05);
}

.category-section {
  padding: 40px 10%;
}

.category-heading {
  font-size: 2rem;
  color: #2c2c2c;
  margin-bottom: 10px;
}

.category-separator {
  border: 0;
  height: 2px;
  background-color: #d4af37;
  margin-bottom: 30px;
  width: 100px;
}

/* The Core Product Grid */
.product-grid {
  display: grid;
  /* Automatically creates responsive columns. Will fit as many 280px wide columns as possible before
  wrapping to the next line. */
  grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
  gap: 30px;
}

/* Individual Product Card */
.product-card {
  background-color: #ffffff;
  border: 1px solid #e0e0e0;
  padding: 20px;
  text-align: center;
  border-radius: 8px;
  box-shadow: 0 4px 6px rgba(0,0,0,0.05);
  transition: transform 0.3s ease, border-color 0.3s ease;
}

.product-card:hover {
  transform: translateY(-5px);
}

```



```

    border-color: #d4af37;
}

/* Product Images */
.product-image {
    width: 100%;
    height: 250px;
    background-color: #e8e6e1;
    object-fit: cover; /* Ensures images fill the box without distorting */
    border-radius: 4px;
    margin-bottom: 15px;
}

/* Product Typography inside the Grid */
.product-title {
    font-size: 1.4rem;
    margin-bottom: 10px;
}

.product-description {
    font-size: 0.9rem;
    color: #666666;
    margin-bottom: 15px;
    height: 60px; /* Keeps descriptions at a uniform height for alignment */
}

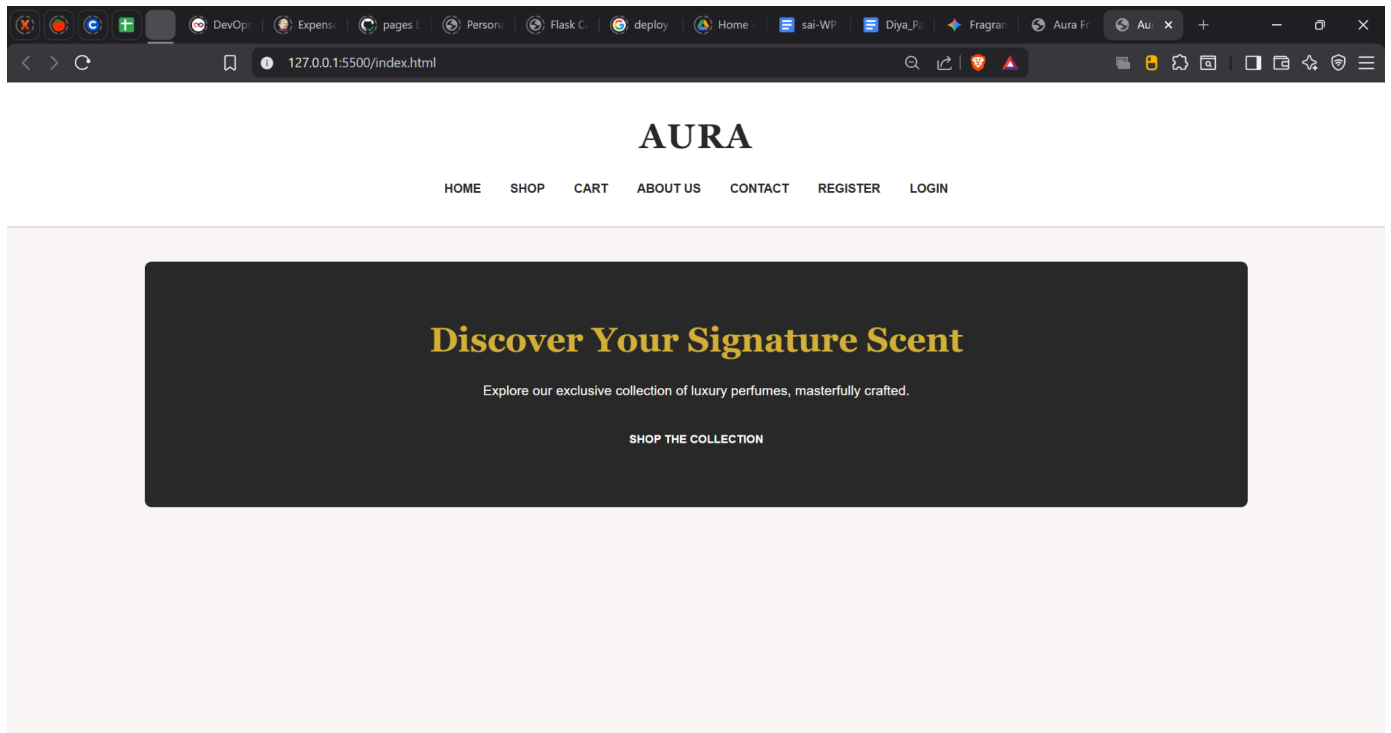
.product-price {
    font-size: 1.2rem;
    font-weight: bold;
    color: #d4af37;
    margin-bottom: 20px;
}

footer {
    text-align: center;
    padding: 20px;
    background-color: #1a1a1a;
    color: #ffffff;
    margin-top: 40px;
}

```

Output:

A. styles.css output:



Code:

B. menu/product page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Shop</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Our Fragrance Collection</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="cart.html">View Cart</a>
    </nav>
  </header>

  <main>
    <section class="category-section">
      <h2 class="category-heading">Women's Collection</h2>
      <hr class="category-separator">

      <div class="product-grid">

        <div class="product-card">
          
          <h3 class="product-title">Midnight Rose</h3>
          <p class="product-description"><strong>Top:</strong> Bergamot <br><strong>Heart:</strong>
Damask Rose <br><strong>Base:</strong> Patchouli, Amber</p>
          <p class="product-price">$125.00</p>
        </div>
      </div>
    </section>
  </main>
</body>
</html>
```

```

        <button class="btn-primary" onclick="addToCart('Midnight Rose', 125.00)">Add to Cart</button>
    </div>

    <div class="product-card">
        
        <h3 class="product-title">Vanilla Silk</h3>
        <p class="product-description"><strong>Top:</strong> Almond <br><strong>Heart:</strong>
Jasmine <br><strong>Base:</strong> Madagascar Vanilla</p>
        <p class="product-price">$110.00</p>
        <button class="btn-primary" onclick="addToCart('Vanilla Silk', 110.00)">Add to Cart</button>
    </div>

</div>
</section>

<section class="category-section">
    <h2 class="category-heading">Men's Collection</h2>
    <hr class="category-separator">

    <div class="product-grid">

        <div class="product-card">
            
            <h3 class="product-title">Oceanic Wood</h3>
            <p class="product-description"><strong>Top:</strong> Sea Salt, Grapefruit
<br><strong>Heart:</strong> Driftwood <br><strong>Base:</strong> Cedar</p>
            <p class="product-price">$95.00</p>
            <button class="btn-primary" onclick="addToCart('Oceanic Wood', 95.00)">Add to Cart</button>
        </div>

        <div class="product-card">
            
            <h3 class="product-title">Smoked Leather</h3>
            <p class="product-description"><strong>Top:</strong> Black Pepper
<br><strong>Heart:</strong> Tobacco Leaf <br><strong>Base:</strong> Rich Leather</p>
            <p class="product-price">$130.00</p>
            <button class="btn-primary" onclick="addToCart('Smoked Leather', 130.00)">Add to
Cart</button>
        </div>

    </div>
</section>
</main>

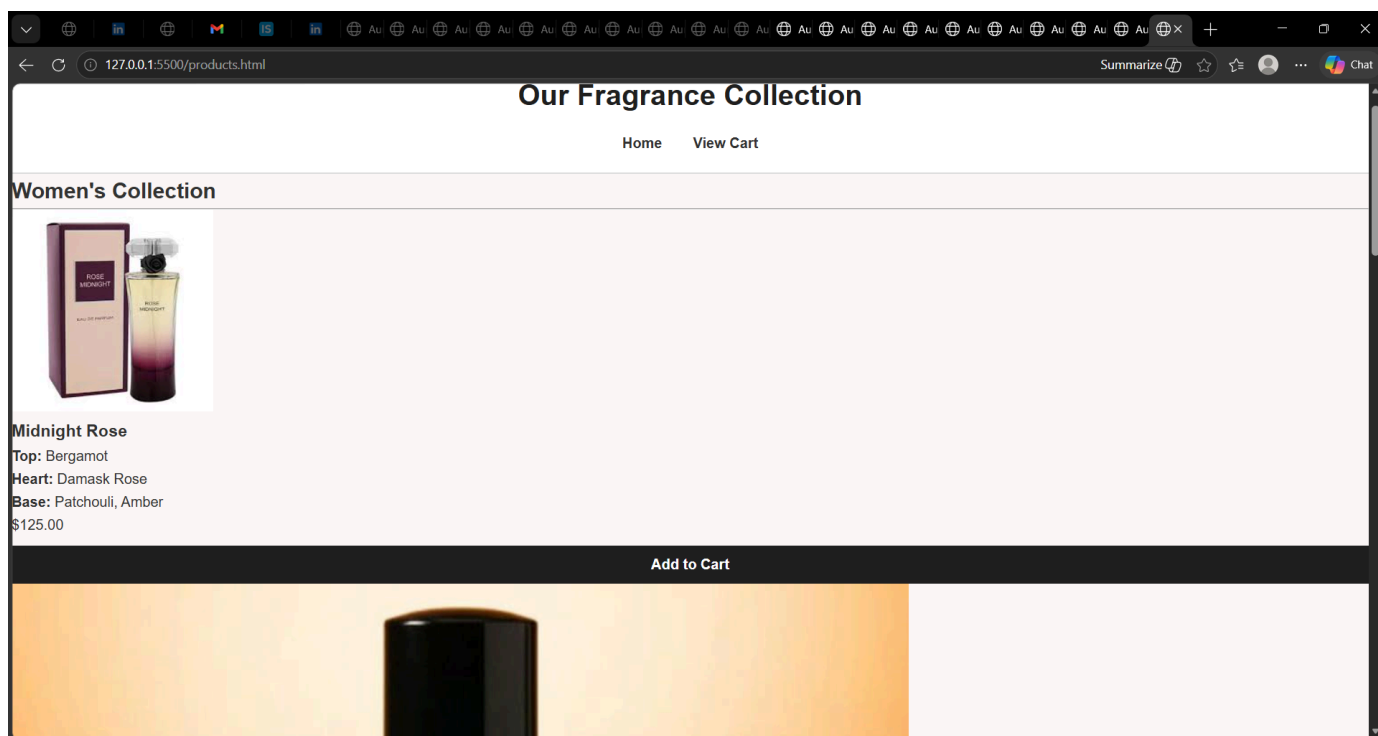
<footer>
    <p>&copy; 2026 Aura Fragrances. All rights reserved.</p>
</footer>

<script src="script.js"></script>
</body>
</html>

```

Output:

B. product page output:



Conclusion: In this experiment, the presentation layer of the application was significantly advanced through the implementation of the CSS Grid Layout module. By defining two-dimensional grid formatting contexts, the product catalog and home page layouts were mathematically structured. The utilization of intrinsic sizing algorithms—specifically combining the `repeat()` function with `auto-fit` and `minmax()`—successfully created a fluid, natively responsive grid. This achievement demonstrated how modern CSS engines can autonomously calculate spatial distribution and adapt to varying viewport sizes without the need for rigid, breakpoint-heavy Media Queries.

Experiment No.4

Problem Statement:

- Enhance the cart page to make it user-friendly and visually appealing. Style the cart items with appropriate margins, paddings, and input field styles to provide a seamless shopping experience.
- Enhance and style the about us page with appropriate margins, paddings, and input field styles.

- Enhance and style the contact page to make it user-friendly and visually appealing. Style the contact form with appropriate margins, paddings, and input field styles.
- Enhance and style the admin/user registration form with appropriate margins, paddings, and input field styles.
- Enhance and style the admin/user login form with appropriate margins, paddings, and input field styles.

Objective:

To apply the CSS Box Model to style user input forms, shopping cart tables, and informational pages, enhancing the visual aesthetics and usability of the fragrance shop.

Theory:

Deep Theoretical Background: While CSS Grid handles macroscopic page layout, the **CSS Box Model** governs the microscopic layout of individual components. Understanding this mathematical model is the foundation of front-end development.

1. Box Model Mathematics: Every HTML element is a rectangular box comprising four concentric layers:

- **Content Area:** The actual text or image dimensions.
- **Padding:** The transparent space between the content and the border. In UI design, increasing padding expands the "hit area" of a button or input field, drastically improving usability on touch devices (Fitts's Law).
- **Border:** The visible boundary wrapping the padding.
- **Margin:** The transparent space outside the border that pushes adjacent elements away, creating necessary whitespace to reduce visual clutter.

2. The box-sizing Paradigm: By default (the W3C standard box model), defining `width: 100%` and adding `padding: 20px` results in an element that is 100% *plus* 40px wide, causing it to break out of its container and create horizontal scrollbars. Applying `box-sizing: border-box;` alters the rendering engine's calculation. It forces the engine to absorb padding and borders *inward*, maintaining the exact specified width.

3. State Management and Visual Feedback: UI elements must communicate their state to the user. CSS pseudo-classes act as state selectors.

- `:hover` activates when a cursor intersects the element's bounding box.
- `:focus` activates when a form element is ready to receive keyboard input. Providing a distinct border color on `:focus` (e.g., gold) is a critical accessibility standard, allowing keyboard-only users to see exactly where they are on the page.

Code:

A. cart page:

Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Aura Fragrances | Shopping Cart</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Your Shopping Cart</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="products.html">Continue Shopping</a>
    </nav>
  </header>

  <main class="cart-container">
    <table class="cart-table">
      <thead>
        <tr>
          <th>Product Details</th>
          <th>Price</th>
          <th>Quantity</th>
          <th>Total</th>
          <th>Action</th>
        </tr>
      </thead>
      <tbody id="cart-tbody">
        <tr>
          <td><strong>Midnight Rose</strong></td>
          <td>$125.00</td>
          <td>
            <input type="number" value="1" min="1" style="width: 60px; padding: 8px; text-align: center;
border: 1px solid #ccc; border-radius: 4px;"/>
          </td>
          <td>$125.00</td>
          <td><button class="btn-remove">Remove</button></td>
        </tr>
      </tbody>
      <tfoot>
        <tr>
          <td colspan="3"><strong>Grand Total:</strong></td>
          <td colspan="2"><strong style="color: #d4af37;">$125.00</strong></td>
        </tr>
      </tfoot>
    </table>

    <div class="cart-actions">
      <button class="btn-primary" style="width: auto;">Proceed to Secure Checkout</button>
    </div>
  </main>

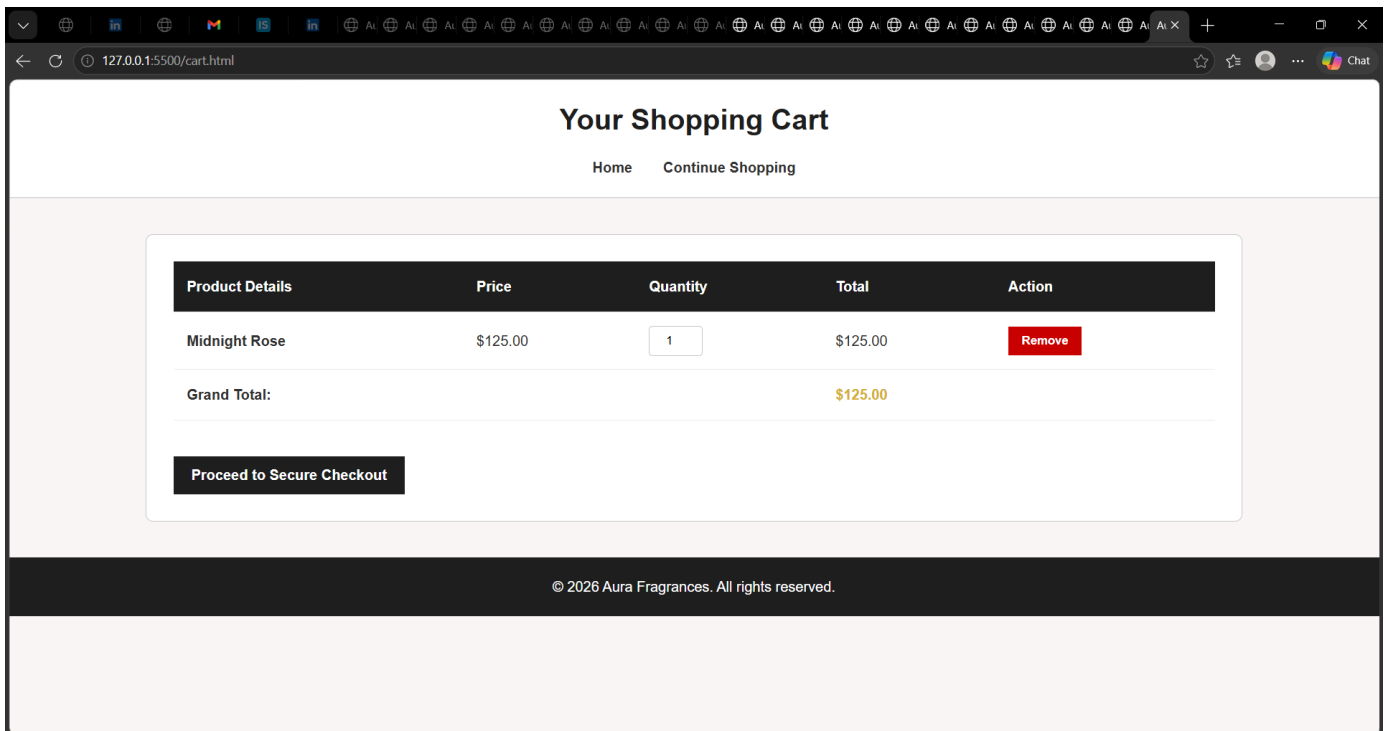
  <footer>
    <p>&copy; 2026 Aura Fragrances. All rights reserved.</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>

```

Output:

A. cart page output:



Code:

B. about us page:

code:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | About Us</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>The Story of Aura</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="products.html">Shop</a>
    </nav>
  </header>

  <main class="about-container">
    <h2>Our Heritage</h2>
    <p>Founded by master perfumers in 2026, Aura Fragrances was established with a singular, passionate vision: to restore true artistry, emotion, and craftsmanship to modern perfumery. We believe that a scent is an invisible, deeply personal signature that leaves a lasting impression.</p>

    <h2>Our Mission & Ethical Promise</h2>

```

```

<p>We are dedicated to delivering unparalleled luxury without compromising our rigorous ethical
standards. Our core philosophies include:</p>
<ul>
  <li><strong>Ethical Sourcing:</strong> We travel the globe to partner directly with farmers who grow
our botanicals sustainably.</li>
  <li><strong>100% Cruelty-Free:</strong> We have never, and will never, test our formulas on
animals.</li>
  <li><strong>Small-Batch Craftsmanship:</strong> Every bottle is hand-blended, macerated, and aged
to perfection in our boutique laboratory.</li>
</ul>

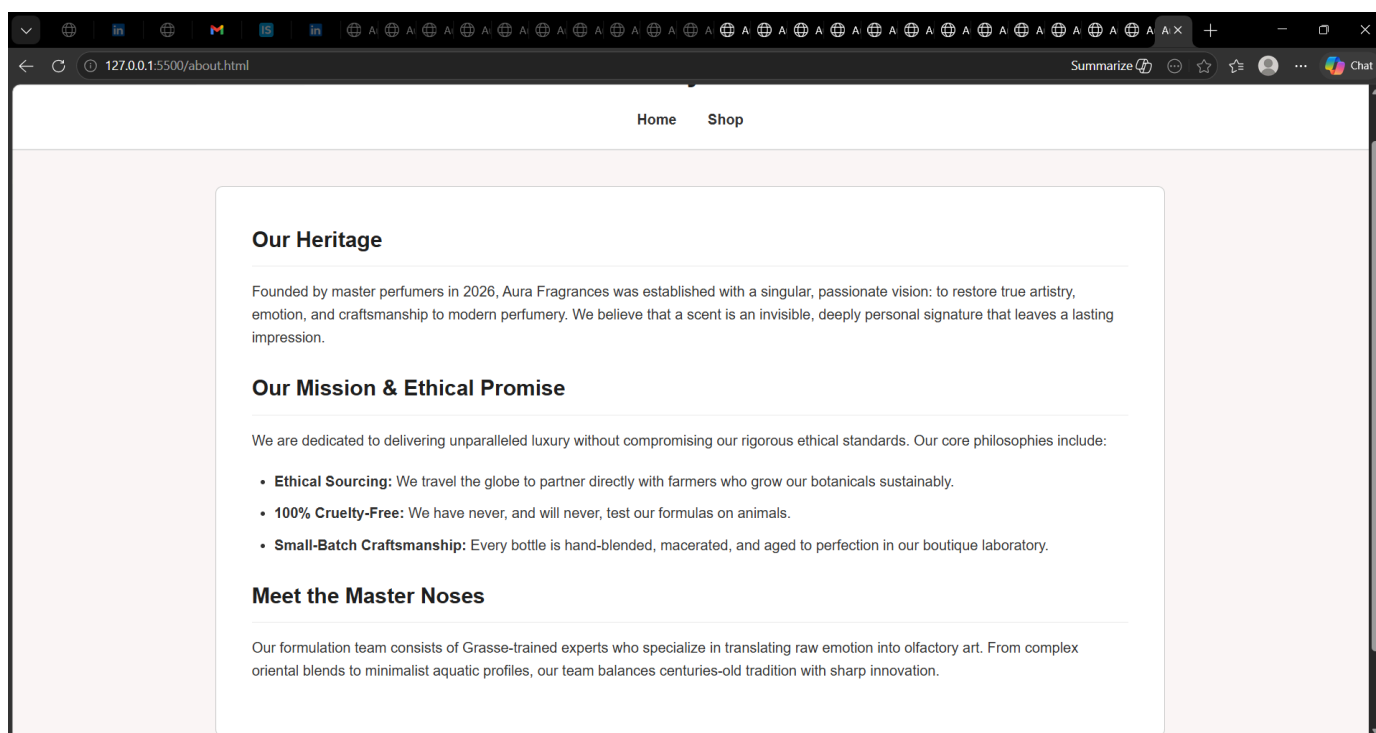
<h2>Meet the Master Noses</h2>
<p>Our formulation team consists of Grasse-trained experts who specialize in translating raw emotion
into olfactory art. From complex oriental blends to minimalist aquatic profiles, our team balances
centuries-old tradition with sharp innovation.</p>
</main>

<footer>
  <p>&copy; 2026 Aura Fragrances.</p>
</footer>
</body>
</html>

```

Output:

B. about us page output:



Code:

C. contact us page:

code:

```

<!DOCTYPE html>
<html lang="en">
<head>

```



```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Aura Fragrances | Contact</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Get in Touch</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="about.html">About Us</a>
    </nav>
  </header>

  <main>
    <div class="form-container">
      <h2>Send us a Message</h2>
      <form action="#" method="POST">
        <div class="form-group">
          <label for="name">Full Name:</label>
          <input type="text" id="name" name="name" placeholder="Enter your full name" required>
        </div>

        <div class="form-group">
          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" placeholder="Enter your email" required>
        </div>

        <div class="form-group">
          <label for="message">Your Message:</label>
          <textarea id="message" name="message" rows="5" placeholder="How can our fragrance experts
assist you?" required></textarea>
        </div>

        <button type="submit" class="btn-primary">Submit Inquiry</button>
      </form>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Aura Fragrances.</p>
  </footer>
</body>
</html>

```

Output:

C. contact us page output:

The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5500/contact.html'. The page has a header with the title 'Get in Touch' and two navigation links: 'Home' and 'About Us'. The main content area features a light pink background with a white form titled 'Send us a Message'. The form contains three input fields: 'Full Name:' with a placeholder 'Enter your full name', 'Email Address:' with a placeholder 'Enter your email', and 'Your Message:' with a placeholder 'How can our fragrance experts assist you?'. Below these fields is a black button labeled 'Submit Inquiry'.

Code:

D. registration page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Register</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Join the Aura Society</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="login.html">Login</a>
    </nav>
  </header>

  <main>
    <div class="form-container">
      <h2>Create Your Account</h2>
      <form id="register-form" action="register.php" method="POST">
        <div class="form-group">
          <label for="fname">Full Name:</label>
          <input type="text" id="fname" name="fname" placeholder="Enter your full name" required>
        </div>

        <div class="form-group">
          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" placeholder="name@example.com" required>
        </div>
      </form>
    </div>
  </main>
</body>
</html>
```

```

</div>

<div class="form-group">
  <label for="password">Create Password:</label>
  <input type="password" id="password" name="password" placeholder="Minimum 8 characters"
required>
</div>

<div class="form-group">
  <label for="confirm_pass">Confirm Password:</label>
  <input type="password" id="confirm_pass" name="confirm_pass" placeholder="Re-enter your
password" required>
</div>

  <button type="submit" class="btn-primary">Register Account</button>
</form>
  <p style="text-align: center; margin-top: 15px;">Already a member? <a href="login.html" style="color:
#d4af37; text-decoration: none; font-weight: bold;">Log in here</a>.</p>
</div>
</main>

<footer>
  <p>&copy; 2026 Aura Fragrances.</p>
</footer>

<script src="script.js"></script>
</body>
</html>

```

Output:

D. registration page output:

The screenshot shows a web browser window with the URL 127.0.0.1:5500/index.html. The page has a navigation bar with 'Home' and 'Login' links. The main content area features a registration form titled 'Create Your Account'. The form includes the following fields and elements:

- Full Name:** A text input field with the placeholder text 'Enter your full name'.
- Email Address:** A text input field with the placeholder text 'name@example.com'.
- Create Password:** A text input field with the placeholder text 'Minimum 8 characters'.
- Confirm Password:** A text input field with the placeholder text 'Re-enter your password'.
- Register Account:** A black button with white text.
- Footer:** A line of text that reads 'Already a member? [Log in here.](#)'.

Code:

E. login page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Aura Fragrances | Login</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Member Login</h1>
    <nav>
      <a href="index.html">Home</a>
      <a href="register.html">Register</a>
    </nav>
  </header>

  <main>
    <div class="form-container">
      <h2>Welcome Back</h2>
      <form id="login-form" action="login.php" method="POST">
        <div class="form-group">
          <label for="login_email">Email Address:</label>
          <input type="email" id="login_email" name="email" placeholder="Enter your registered email"
required>
        </div>

        <div class="form-group">
          <label for="login_password">Password:</label>
          <input type="password" id="login_password" name="password" placeholder="Enter your
password" required>
        </div>

        <button type="submit" class="btn-primary">Sign In</button>
      </form>
      <p style="text-align: center; margin-top: 15px;">New to Aura? <a href="register.html" style="color:
#d4af37; text-decoration: none; font-weight: bold;">Create an account</a>.</p>
    </div>
  </main>

  <footer>
    <p>&copy; 2026 Aura Fragrances.</p>
  </footer>

  <script src="script.js"></script>
</body>
</html>
```

Output:

E. login page output:

The screenshot shows a web browser window displaying a 'Member Login' page. The browser's address bar shows '127.0.0.1:5500/login.html'. The page has a light purple header with the title 'Member Login' and navigation links 'Home' and 'Register'. The main content area is a light pink box containing a 'Welcome Back' message, a prompt 'Please sign in to access your account.', and a login form. The form has two input fields: 'Email Address' with a placeholder 'Enter your registered email' and 'Password' with a placeholder 'Enter your password'. Below the fields is a black 'Sign In' button. At the bottom of the form, there is a link 'New to Aura? Create an account.'.

Conclusion:

This experiment successfully applied the CSS Box Model to polish the User Interface, proving that precise control over margins, borders, and padding is essential for a high-quality user experience. By expanding internal padding on form inputs and buttons, the application adhered to Fitts's Law, increasing hit-areas for better accessibility on touch devices. Applying `box-sizing: border-box` ensured mathematical layout stability. Furthermore, the implementation of pseudo-classes (`:hover` and `:focus`) introduced critical state-based visual feedback, ensuring the application communicates interactively with the user during data entry and navigation.

Experiment No.5

Problem Statement:

- Implement user registration and login forms for the Fragrance shop website. These forms will allow users to create an account, log in, and access personalized features, such as saving favorite items or viewing order history.
 - User Registration Form will allow new customers to sign up and create an account on the website. The form will capture basic user details, including the name, email address, and password (not limited to these fields).
 - User Login Form will allow registered users to log into their accounts. The form will require an email address and a password to authenticate the user.
- Provide validations for user registration and login forms to validate the input to ensure that all required fields are filled and that the email format is valid. **(Contents beyond Syllabus)**
- Develop cart functionality to allow users to add items, update quantities, and remove items.

Objective:

To integrate client-side JavaScript for interactive form validation and dynamic shopping cart manipulation on the fragrance shop website.

Theory:

Deep Theoretical Background: JavaScript is a single-threaded, non-blocking, asynchronous programming language. In the context of a browser, it interacts directly with the DOM to manipulate the page dynamically without requiring a server reload.

1. The Event-Driven Programming Model: JavaScript operates on an Event Loop. The browser listens for user interactions (clicks, keystrokes, form submissions) and places them in an event queue. When a user submits a registration form, we attach an `addEventListener` to intercept the `submit` event. Calling `event.preventDefault()` stops the browser's default behavior (sending a synchronous HTTP request and reloading the page), allowing our script to take control.

2. Client-Side Validation and Regular Expressions (Regex): Validating data on the client side drastically improves UX by providing instantaneous feedback and reduces unnecessary server load. To validate an email, we use **Regular Expressions (Regex)**, which are sequences of characters defining a search pattern based on automata theory. A pattern like `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` programmatically ensures the string contains text, followed by exactly one `@` symbol, followed by a domain, and a dot.

3. State Management via Arrays: The shopping cart requires state management. We utilize a JavaScript Array of Objects. Each object represents a perfume entity. When a user adds an item, the script executes an algorithm: it iterates through the array (`Array.prototype.find`) to check if the item ID already exists. If `true`, it mutates the object by incrementing the quantity. If `false`, it uses `Array.prototype.push` to append a new object. The script then dynamically generates HTML strings and injects them into the DOM via the `innerHTML` property, forcing the browser to repaint the cart table.

Code:

A. script.js:

code:

```
document.addEventListener("DOMContentLoaded", function() {

    const registerForm = document.getElementById("register-form");

    if (registerForm) {
        registerForm.addEventListener("submit", function(event) {
            event.preventDefault(); // Stop page from reloading

            // Get the text from the inputs
            const fname = document.getElementById("fname").value.trim();
            const email = document.getElementById("email").value.trim();
            const password = document.getElementById("password").value;
            const confirmPass = document.getElementById("confirm_pass").value;

            // Check 1: Are any fields empty?
            if (fname === "" || email === "" || password === "" || confirmPass === "") {
                alert("Please fill in all the required fields.");
                return; // Stop the function here
            }
        });
    }
});
```

```

// Check 2: Is the email format valid?
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
    alert("Please enter a valid email address.");
    return;
}

// Check 3: Do the passwords match?
if (password !== confirmPass) {
    alert("Your passwords do not match. Please try again.");
    return;
}

// Save the user data to the browser's local storage
const userData = { name: fname, email: email, password: password };
localStorage.setItem("aura_registered_user", JSON.stringify(userData));

alert("Registration successful! You can now log in.");
window.location.href = "login.html"; // Send user to login page
});
}

const loginForm = document.getElementById("login-form");

if (loginForm) {
    loginForm.addEventListener("submit", function(event) {
        event.preventDefault(); // Stop page from reloading

        const email = document.getElementById("login_email").value.trim();
        const password = document.getElementById("login_password").value;

        // Check 1: Are fields empty?
        if (email === "" || password === "") {
            alert("Please enter both your email and password.");
            return;
        }

        // Get the saved user from local storage
        const savedUserStr = localStorage.getItem("aura_registered_user");

        if (savedUserStr) {
            const savedUser = JSON.parse(savedUserStr);

            // Check 2: Do the credentials match what is saved?
            if (email === savedUser.email && password === savedUser.password) {
                alert("Login successful! Welcome back, " + savedUser.name + ".");
                window.location.href = "index.html"; // Send user to home page
            } else {
                alert("Incorrect email or password.");
            }
        } else {
            alert("No account found. Please register first.");
        }
    });
}

// Run the cart display function if we are on the cart page

```

```

    renderCart();
  });

  // Load the cart from memory, or start an empty array
  let cart = JSON.parse(localStorage.getItem("aura_cart")) || [];

  function saveCart() {
    localStorage.setItem("aura_cart", JSON.stringify(cart));
  }

  function addToCart(productName, price) {
    let existingItem = cart.find(item => item.name === productName);

    if (existingItem) {
      existingItem.quantity += 1;
    } else {
      cart.push({ name: productName, price: price, quantity: 1 });
    }

    saveCart();
    alert(productName + " added to cart!");
    renderCart();
  }

  function updateQuantity(index, newQuantity) {
    if (newQuantity <= 0) {
      removeFromCart(index);
    } else {
      cart[index].quantity = newQuantity;
      saveCart();
      renderCart();
    }
  }

  function removeFromCart(index) {
    cart.splice(index, 1);
    saveCart();
    renderCart();
  }

  function renderCart() {
    const tbody = document.getElementById("cart-tbody");
    if (!tbody) return;

    tbody.innerHTML = "";
    let grandTotal = 0;

    for (let i = 0; i < cart.length; i++) {
      let item = cart[i];
      let itemTotal = item.price * item.quantity;
      grandTotal += itemTotal;

      let tr = document.createElement("tr");
      tr.innerHTML = `
        <td>${item.name}</td>
        <td>${item.price.toFixed(2)}</td>
        <td>

```



```

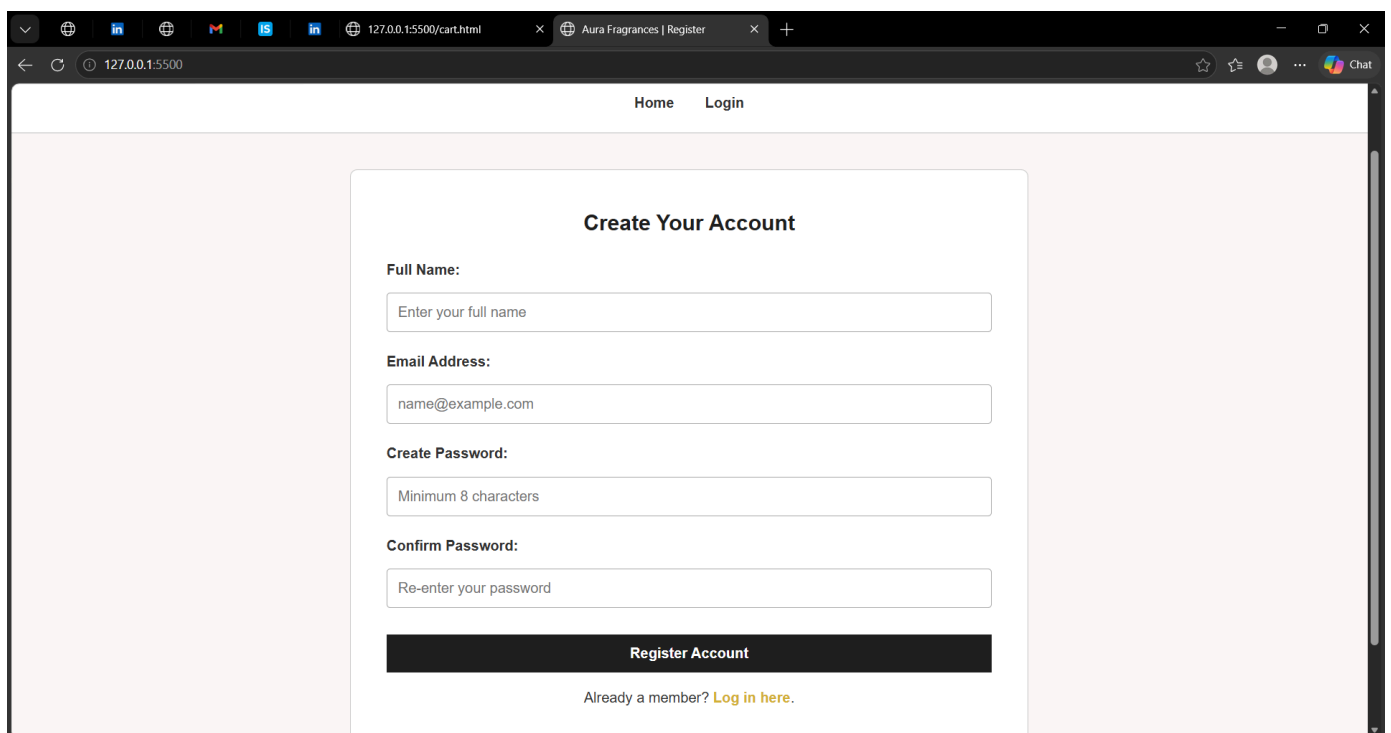
        <input type="number" value="${item.quantity}" min="1" onchange="updateQuantity(${i},
parseInt(this.value))">
    </td>
    <td>${itemTotal.toFixed(2)}</td>
    <td><button onclick="removeFromCart(${i})">Remove</button></td>
    `;
    tbody.appendChild(tr);
}

const totalElements = document.querySelector("tfoot strong");
if (totalElements.length > 0) {
    totalElements[totalElements.length - 1].innerText = "$" + grandTotal.toFixed(2);
}
}
}

```

Output:

A. Index page output:



Code:

B. registration page:

code:

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Aura Fragrances | Register</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>

```

```

<h1>Join the Aura Society</h1>
<nav>
  <a href="index.html">HOME</a>
  <a href="login.html">LOGIN</a>
</nav>
</header>

<main class="auth-main">
  <div class="auth-card">
    <h2>Create Your Account</h2>
    <form id="register-form">
      <div class="input-group">
        <label for="fname">Full Name:</label>
        <input type="text" id="fname" placeholder="Enter your full name" required>
      </div>

      <div class="input-group">
        <label for="email">Email Address:</label>
        <input type="email" id="email" placeholder="name@example.com" required>
      </div>

      <div class="input-group">
        <label for="password">Create Password:</label>
        <input type="password" id="password" placeholder="Minimum 8 characters" required>
      </div>

      <div class="input-group">
        <label for="confirm_pass">Confirm Password:</label>
        <input type="password" id="confirm_pass" placeholder="Re-enter your password" required>
      </div>

      <button type="submit" class="auth-btn">REGISTER ACCOUNT</button>
    </form>

    <p class="auth-link">Already a member? <a href="login.html">Log in here</a>.</p>
  </div>
</main>

<script src="script.js"></script>
</body>
</html>

```

Output:

B. registration page output:

Join the Aura Society

HOME LOGIN

Create Your Account

Full Name:

Enter your full name

Email Address:

name@example.com

Create Password:

Minimum 8 characters

Confirm Password:

Re-enter your password

[REGISTER ACCOUNT](#)

Already a member? [Log in here](#).

Code:

C. login page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Aura Fragrances | Login</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Member Login</h1>
    <nav>
      <a href="index.html">HOME</a>
      <a href="register.html">REGISTER</a>
    </nav>
  </header>

  <main class="auth-main">
    <div class="auth-card">
      <h2>Welcome Back</h2>
      <form id="login-form">
        <div class="input-group">
          <label for="login_email">Email Address:</label>
          <input type="email" id="login_email" placeholder="Enter your registered email" required>
        </div>

        <div class="input-group">
          <label for="login_password">Password:</label>
          <input type="password" id="login_password" placeholder="Enter your password" required>
        </div>
      </form>
    </div>
  </main>
</body>
</html>
```

```

        <button type="submit" class="auth-btn">SIGN IN</button>
    </form>

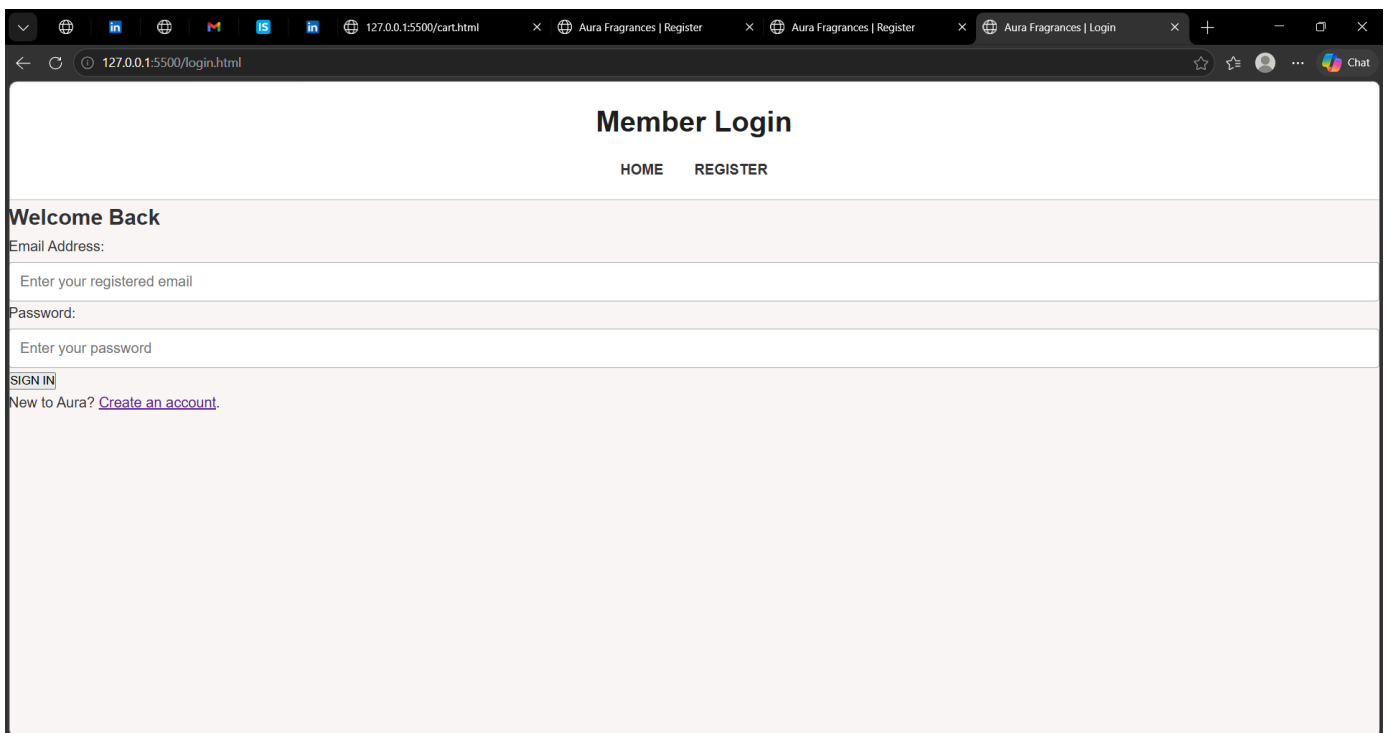
    <p class="auth-link">New to Aura? <a href="register.html">Create an account</a>.</p>
</div>
</main>

<script src="script.js"></script>
</body>
</html>

```

Output:

C. login page output:



Conclusion:

In this experiment, client-side interactivity and event-driven programming were successfully integrated using JavaScript. The implementation of client-side form validation intercepted HTTP submit events to enforce data integrity—utilizing Regular Expressions (Regex) to verify email syntax and logical operators to ensure password matches—thereby improving UX and reducing unnecessary server requests. Additionally, the development of a dynamic shopping cart demonstrated in-memory state management. By utilizing an Array of Objects and array manipulation methods (**push**, **splice**), the script successfully tracked product data and dynamically re-rendered the DOM to reflect real-time financial subtotals and quantities.

Experiment No.6

Problem Statement:

- The user login form will allow registered users to log into their accounts. The form will require an email address and a password to authenticate the user.
- If the login is successful, the user should be redirected to the homepage or their user dashboard.
- Use localStorage or sessionStorage to store authentication data, such as the user's email and login status. This ensures that once a user is logged in, they remain authenticated even after the page reloads or when they visit the site again.
- Save the cart data to local storage when items are added, updated, or removed. Retrieve and load the cart data from local storage when the page loads.

Objective:

To utilize the Web Storage API (**localStorage**) to persist user authentication status and shopping cart data across browser sessions and page reloads.

Theory:

Deep Theoretical Background: The fundamental architecture of the World Wide Web relies on the **HTTP Protocol**, which is inherently **stateless**. The server treats every request as completely independent. If a user adds an item to their cart on **products.html** and navigates to **cart.html**, the browser forgets the JavaScript array, resulting in an empty cart.

1. The Web Storage Solution: To solve client-side persistence, HTML5 introduced the Web Storage API, which provides mechanisms to store key-value pairs securely in the browser's memory.

- **sessionStorage**: Maintains a separate storage area for the duration of the page session (until the browser tab is closed).
- **localStorage**: Persists data indefinitely across browser sessions. It survives page reloads and browser restarts, making it the ideal architectural choice for a guest shopping cart.

2. Data Serialization and Deserialization (JSON): A critical limitation of the Web Storage API is that it can **only store string primitives**. It cannot store complex data types like JavaScript Objects or Arrays directly in memory. To bypass this, we use **Serialization**. We pass our cart array through the `JSON.stringify()` algorithm, which converts the entire data structure into a single JSON (JavaScript Object Notation) formatted text string. When we load the page, we retrieve this string from memory and perform **Deserialization** using `JSON.parse()`, which evaluates the string and reconstructs the functional JavaScript array in RAM.

Code:

A. registration page:

code:

```
document.addEventListener("DOMContentLoaded", function() {

    // --- A. SESSION MANAGEMENT: Check if user is already logged in ---
    const activeSessionStr = localStorage.getItem("aura_active_session");

    // Optional UI enhancement: Change "LOGIN" to "LOGOUT" in the nav if logged in
    const navLinks = document.querySelectorAll("nav a");
    if (activeSessionStr) {
        navLinks.forEach(link => {
            if (link.innerText.toUpperCase() === "LOGIN" || link.innerText.toUpperCase() === "REGISTER") {
                link.innerText = "LOGOUT";
                link.href = "#"; // Prevent default navigation
                link.addEventListener("click", function(e) {
                    e.preventDefault();
                    localStorage.removeItem("aura_active_session"); // Destroy session
                    alert("You have been successfully logged out.");
                    window.location.href = "login.html"; // Redirect to login
                });
            }
        });
    }

    // --- B. USER REGISTRATION LOGIC ---
    const registerForm = document.getElementById("register-form");

    if (registerForm) {
        registerForm.addEventListener("submit", function(event) {
            event.preventDefault();

            const fname = document.getElementById("fname").value.trim();
            const email = document.getElementById("email").value.trim();
            const password = document.getElementById("password").value;
            const confirmPass = document.getElementById("confirm_pass").value;

            if (fname === "" || email === "" || password === "" || confirmPass === "") {
                alert("Validation Error: All fields are required.");
                return;
            }

            if (password !== confirmPass) {
                alert("Validation Error: Passwords do not match.");
                return;
            }
        });
    }
});
```

```

    }

    // Save user credentials to LocalStorage (Simulating Database)
    const userData = { name: fname, email: email, password: password };
    localStorage.setItem("aura_registered_user", JSON.stringify(userData));

    alert("Registration successful! Welcome to the Aura Society, " + fname + ".");
    window.location.href = "login.html";
  });
}

// --- C. USER LOGIN LOGIC ---
const loginForm = document.getElementById("login-form");

if (loginForm) {
  // If they visit the login page but are already logged in, redirect them away
  if (activeSessionStr) {
    window.location.href = "index.html";
  }

  loginForm.addEventListener("submit", function(event) {
    event.preventDefault();

    const email = document.getElementById("login_email").value.trim();
    const password = document.getElementById("login_password").value;

    const savedUserStr = localStorage.getItem("aura_registered_user");

    if (savedUserStr) {
      const savedUser = JSON.parse(savedUserStr);

      if (email === savedUser.email && password === savedUser.password) {

        // EXPERIMENT 6: Store Authentication Data (Session)
        const sessionData = {
          email: savedUser.email,
          name: savedUser.name,
          isLoggedIn: true,
          loginTime: new Date().getTime()
        };
        localStorage.setItem("aura_active_session", JSON.stringify(sessionData));

        alert("Welcome back, " + savedUser.name + "! Login successful.");

        // EXPERIMENT 6: Redirect to homepage on success
        window.location.href = "index.html";
      } else {
        alert("Login Error: Incorrect email or password.");
      }
    } else {
      alert("Login Error: No registered account found. Please register first.");
    }
  });
}

// Render the cart on page load (Retrieves data from LocalStorage)
renderCart();

```

```

});

// Retrieve cart data from local storage on page load. If empty, start a new array.
let cart = JSON.parse(localStorage.getItem("aura_cart")) || [];

// Helper function to save the cart array back to LocalStorage
function saveCartToStorage() {
  localStorage.setItem("aura_cart", JSON.stringify(cart));
}

// Function 1: Add Items to Cart
function addToCart(productName, price) {
  let existingItem = cart.find(item => item.name === productName);

  if (existingItem) {
    existingItem.quantity += 1;
  } else {
    cart.push({ name: productName, price: price, quantity: 1 });
  }

  saveCartToStorage(); // Save immediately to LocalStorage
  alert(productName + " has been added to your cart.");
  renderCart(); // Update UI
}

// Function 2: Update Quantities
function updateQuantity(index, newQuantity) {
  if (newQuantity <= 0) {
    removeFromCart(index);
  } else {
    cart[index].quantity = newQuantity;
    saveCartToStorage(); // Save immediately to LocalStorage
    renderCart(); // Update UI
  }
}

// Function 3: Remove Items
function removeFromCart(index) {
  cart.splice(index, 1);
  saveCartToStorage(); // Save immediately to LocalStorage
  renderCart(); // Update UI
}

// Function 4: Render Cart to the HTML Table
function renderCart() {
  const tbody = document.getElementById("cart-tbody");
  if (!tbody) return;

  tbody.innerHTML = "";
  let grandTotal = 0;

  cart.forEach((item, index) => {
    let itemTotal = item.price * item.quantity;
    grandTotal += itemTotal;

    let tr = document.createElement("tr");
    tr.innerHTML = `

```



```

<td style="padding: 15px 10px; border-bottom: 1px solid #eee;"><strong>${item.name}</strong></td>
<td style="padding: 15px 10px; border-bottom: 1px solid #eee;">${item.price.toFixed(2)}</td>
<td style="padding: 15px 10px; border-bottom: 1px solid #eee;">
  <input type="number" value="${item.quantity}" min="1"
    onchange="updateQuantity(${index}, parseInt(this.value))"
    style="width: 60px; padding: 8px; text-align: center; border: 1px solid #ccc; border-radius: 4px;">
</td>
<td style="padding: 15px 10px; border-bottom: 1px solid #eee;">${itemTotal.toFixed(2)}</td>
<td style="padding: 15px 10px; border-bottom: 1px solid #eee;">
  <button class="btn-remove" style="background-color: #dc3545; color: white; border: none; padding:
8px 12px; border-radius: 4px; cursor: pointer;" onclick="removeFromCart(${index})">Remove</button>
</td>
`;
tbody.appendChild(tr);
});

const grandTotalElements = document.querySelectorAll("tfoot strong");
if (grandTotalElements.length > 0) {
  grandTotalElements[grandTotalElements.length - 1].innerText = "$" + grandTotal.toFixed(2);
}

if (cart.length === 0) {
  tbody.innerHTML = `<tr><td colspan="5" style="text-align: center; padding: 30px; color: #666;">Your cart
is currently empty. Start shopping!</td></tr>`;
  if (grandTotalElements.length > 0) {
    grandTotalElements[grandTotalElements.length - 1].innerText = "$0.00";
  }
}
}
}

```

Output:

A. Index/Home page output:

The screenshot displays a web browser window with multiple tabs open, including '127.0.0.1:5500/cart.html' and 'Aura Fragrances | Register'. The main content area shows a registration form with the following elements:

- Full Name:** A text input field containing 'DIYA'.
- Email Address:** A text input field containing 'diya@test.com'.
- Create Password:** A password input field with masked characters '.....'.
- Confirm Password:** A password input field with masked characters '.....'.
- Register Account:** A prominent yellow button at the bottom of the form.
- Footer:** A link that says 'Already a member? [Log in here.](#)'

A dark overlay message box at the top center of the form area states: '127.0.0.1:5500 says Registration successful! Welcome to the Aura Society, DIYA.' with an 'OK' button.

Conclusion:

This experiment successfully bridged the gap between stateless HTTP navigation and persistent client-side state by utilizing the HTML5 Web Storage API. By implementing `localStorage`, the application achieved data persistence that survives page reloads and browser closures. A key technical achievement in this module was the successful application of data serialization; because the Web Storage API only accepts string primitives, the JavaScript cart array was programmatically converted to a JSON string using `JSON.stringify()` for storage, and safely deserialized back into an array using `JSON.parse()` upon retrieval.

Experiment No.7

Problem Statement:

- Develop a PHP script to handle user registration for the Fragrance shop website. The script should accept input from users for their name, email address, password, etc. (all required fields for registration).
- Implement error handling to notify users of any issues during registration, such as validation errors.
- Provide feedback to the user upon successful registration, either through a confirmation message or a redirect to a login page.

Objective:

To develop a server-side PHP script that securely handles new user registration, validates input data on the server, and provides appropriate user feedback.

Theory:

Deep Theoretical Background: While client-side technologies (HTML/CSS/JS) govern the user interface, **Server-Side Scripting** governs data security, business logic, and permanent storage. PHP (Hypertext Preprocessor) operates exclusively on the web server.

1. The Client-Server Request Cycle: When a user submits the registration form, the browser packages the data into an HTTP POST request and sends it over the network. The web server (e.g., Apache) receives this request and hands it to the PHP runtime environment. PHP populates the `$_POST` superglobal array with the incoming data.

2. Data Sanitization and Security Vulnerabilities: Client-side validation is easily bypassed using developer tools. Therefore, the server operates on a "Zero-Trust Architecture." Incoming data must be sanitized to prevent **Cross-Site Scripting (XSS)**—an attack where malicious scripts are injected into the database and executed on other users' browsers. Functions like `filter_var()` strip out illegal characters before processing.

3. Cryptographic Hashing Algorithms: Storing passwords as plain text is the most severe security failure in software engineering. PHP addresses this using the `password_hash()` function. Hashing is a one-way

cryptographic mathematical operation. Unlike encryption, a hash cannot be "decrypted" back to the original password. Modern algorithms (like Bcrypt) also automatically generate a cryptographic "Salt"—a random string appended to the password before hashing. This defends against "Rainbow Table" attacks (precomputed hash lookups), ensuring that even if two users have the password "perfume123", their database hashes will look completely different.

Code:

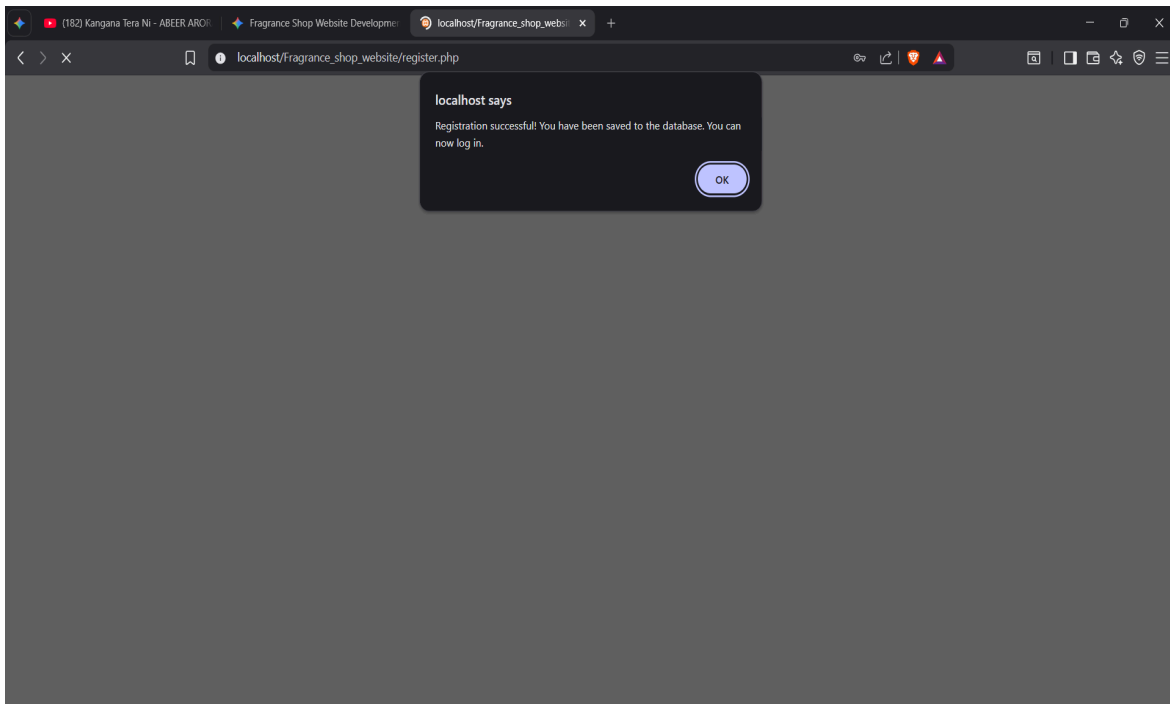
A. registration page:

code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Aura Fragrances | Register</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <header>
    <h1>Join the Aura Society</h1>
    <nav><a href="index.html">HOME</a> | <a href="login.html">LOGIN</a></nav>
  </header>
  <main class="auth-main">
    <div class="auth-card">
      <h2>Create Your Account</h2>
      <form action="register.php" method="POST">
        <div class="input-group">
          <label for="fname">Full Name:</label>
          <input type="text" id="fname" name="fname" required>
        </div>
        <div class="input-group">
          <label for="email">Email Address:</label>
          <input type="email" id="email" name="email" required>
        </div>
        <div class="input-group">
          <label for="password">Create Password:</label>
          <input type="password" id="password" name="password" required>
        </div>
        <div class="input-group">
          <label for="confirm_pass">Confirm Password:</label>
          <input type="password" id="confirm_pass" name="confirm_pass" required>
        </div>
        <button type="submit" class="auth-btn">REGISTER ACCOUNT</button>
      </form>
      <p class="auth-link">Already a member? <a href="login.html">Log in here</a>.</p>
    </div>
  </main>
  <script src="script.js"></script>
</body>
</html>
```

Output:

A. registration page output:



Conclusion:

In this experiment, the application successfully transitioned from front-end interface design to back-end server logic using PHP. The script effectively demonstrated how to capture and process HTTP POST payloads via the `$_POST` superglobal. Crucially, this module highlighted the necessity of a "Zero-Trust" server architecture by implementing server-side data sanitization and validation to mitigate vulnerabilities like Cross-Site Scripting (XSS). Furthermore, the application adhered to modern cybersecurity standards by utilizing the `password_hash()` function, ensuring that user passwords are transformed into secure, salted cryptographic hashes prior to database insertion.

Experiment No.8

Problem Statement:

- Develop a PHP script to handle user login for the Fragrance shop website. The script should accept input from users for their login credentials. (all required fields for login).
- Provide feedback to the user upon successful login, either through a confirmation message or a redirect to a welcome page.
- Implement error handling to notify users of login failures due to incorrect credentials or other errors.
- Provide feedback to the user upon successful login, either through a welcome user name message or a redirect to a home page.

Objective:

To implement a secure PHP login system utilizing server-side sessions to authenticate users and manage their logged-in state across the fragrance shop.

Theory:

Deep Theoretical Background: Because HTTP is stateless, the server cannot naturally identify a user moving from page to page. **Session Management** is the architectural pattern used to build a stateful application layer on top of a stateless protocol.

1. The Mechanics of PHP Sessions: When the PHP `session_start()` function is invoked, the server executes a multi-step protocol:

1. The server generates a highly random, unique **Session ID** (e.g., `PHPSESSID=1a2b3c4d5e`).
2. The server sends this ID back to the user's browser via an HTTP Response Header, instructing the browser to save it as a **Cookie**.
3. Simultaneously, the server creates a secure file in its own temporary directory named after that exact ID.

2. Authentication vs. Authorization:

- **Authentication** is verifying identity. When a user logs in, PHP fetches their hashed password from the database. It uses `password_verify()` to hash the user's input and compares the two hashes. If they match, the user is authenticated. PHP then writes data (like `user_email`) into the `$_SESSION` array, which writes to the server-side file.
- **Authorization** is verifying access rights. On restricted pages, PHP reads the cookie from the browser, finds the matching session file, and checks if `$_SESSION['user_email']` exists. If it does not, the script issues an HTTP 302 Redirect to force the user back to the login page, effectively securing the route.

Code:

A. login page:

code:

```

<?php
session_start();
require 'db.php'; // This connects to your MySQL database

// Check if the form was submitted
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Capture and clean the email input to prevent SQL injection
    $email = $conn->real_escape_string(trim($_POST['email']));
    $password = $_POST['password'];

    // 1. Check if the user exists in the database
    $sql = "SELECT * FROM users WHERE email = '$email'";
    $result = $conn->query($sql);

    if ($result->num_rows > 0) {
        // 2. User exists! Fetch their data as an associative array
        $user = $result->fetch_assoc();

        // 3. Verify the typed password against the hashed password in the database
        if (password_verify($password, $user['password'])) {

            // Success! Create the session variables
            $_SESSION['logged_in'] = true;
            $_SESSION['user_fname'] = $user['fname'];
            $_SESSION['user_email'] = $user['email'];

            // Redirect to the home page with a welcome message
            echo "<script>
                alert('Login successful! Welcome back, " . $user['fname'] . "');
                window.location.href='index.php';
            </script>";
            exit();

        } else {
            // Password did not match
            echo "<script>alert('Error: Incorrect password. Please try again.');" . window.history.back();</script>";
            exit();
        }
    } else {
        // Email was not found in the database
        echo "<script>alert('Error: No account found with that email. Please register first.');" . window.history.back();</script>";
        exit();
    }
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Aura Fragrances | Login</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>Member Login</h1>

```

```

<nav>
  <a href="index.php">HOME</a>
  <a href="register.php">REGISTER</a>
</nav>
</header>

<main class="auth-main">
  <div class="auth-card">
    <h2>Welcome Back</h2>
    <form action="login.php" method="POST">
      <div class="input-group">
        <label for="login_email">Email Address:</label>
        <input type="email" id="login_email" name="email" placeholder="Enter your registered email"
required>
      </div>

      <div class="input-group">
        <label for="login_password">Password:</label>
        <input type="password" id="login_password" name="password" placeholder="Enter your
password" required>
      </div>

      <button type="submit" class="auth-btn">SIGN IN</button>
    </form>

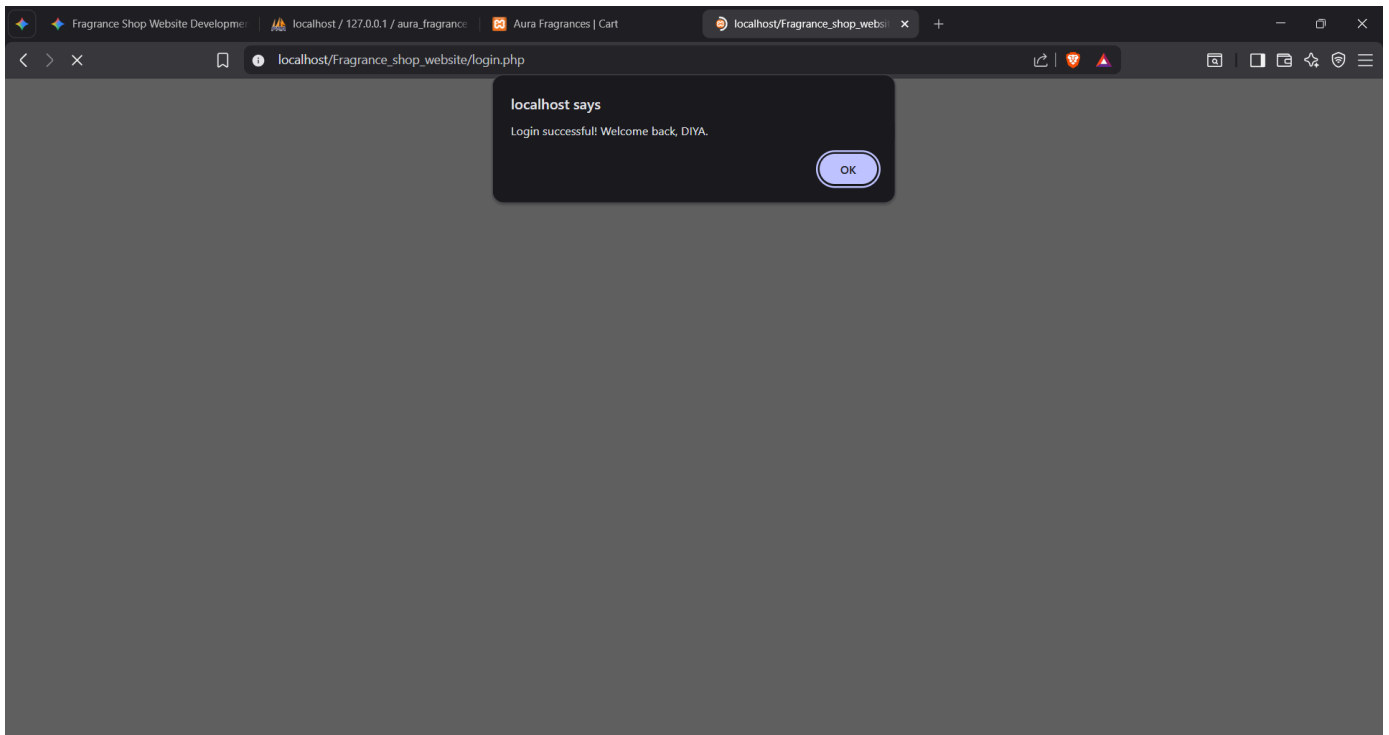
    <p class="auth-link">New to Aura? <a href="register.php">Create an account</a>.</p>
  </div>
</main>

<footer>
  <p>&copy; 2026 Aura Fragrances.</p>
</footer>
<script src="script.js"></script>
</body>
</html>

```

Output:

A. login page:



Conclusion:

This experiment successfully implemented a secure authentication and authorization workflow, overcoming the inherent statelessness of the HTTP protocol. By utilizing PHP Sessions (`session_start()`), the server successfully generated and tracked unique session IDs via client-side cookies. The script executed secure identity verification by retrieving database records and comparing user input against stored hashes using `password_verify()`. The successful assignment of state variables to the `$_SESSION` superglobal demonstrated how modern web applications maintain a persistent "logged-in" state, allowing for secure route protection and personalized user experiences across multiple page requests.

Experiment No.9

Problem Statement:

- Develop a PHP script that allows users to manage their shopping cart for an e-commerce website (e.g., a Fragrance shop store). The script should allow users to add items to their cart, view their cart contents, and remove items if needed.
- Develop a PHP script to manage the shopping cart for an e-commerce website (e.g., a Fragrance shop store) using MySQL. This script should allow users to add items to their cart, view their cart contents, and remove items from the cart. The cart data should be stored in the MySQL database to allow persistence across sessions.

Objective:

To integrate a MySQL database using PHP (**mysqli**) to persistently store, update, and retrieve the user's shopping cart data securely on the server.

Theory:

For robust e-commerce, shopping carts must be stored persistently on the server rather than just in the browser. This requires integrating PHP with a **Relational Database Management System (RDBMS)** like MySQL.

1. **Database Connection:** PHP connects to the MySQL server using the **mysqli** (MySQL Improved) extension or PDO, requiring credentials (hostname, username, password, database name).
2. **CRUD Operations:** Interactions with the database are handled using **SQL (Structured Query Language)**.
 - **Create:** **INSERT INTO** adds a new product to the cart table.
 - **Read:** **SELECT** retrieves the current cart contents for the user.
 - **Update:** **UPDATE** modifies the quantity of an existing item.
 - **Delete:** **DELETE** removes an item from the cart.
3. **Session Tracking:** To associate cart items with a specific user (even if they are not logged in), the PHP Session ID is stored in the database alongside the product details.

Code:

A. Product page:

code:

```
<?php
session_start();
require 'db.php'; // Connects to your database

// Handle the "Add to Cart" button click
if ($_SERVER['REQUEST_METHOD'] == 'POST' && isset($_POST['add_to_cart'])) {
    $session_id = session_id();
    $product_name = $conn->real_escape_string($_POST['product_name']);
    $price = (float)$_POST['price'];

    // Check if item already exists in the cart for this session
    $check_sql = "SELECT * FROM user_cart WHERE session_id = '$session_id' AND product_name = '$product_name'";
```

```

$result = $conn->query($check_sql);

if ($result->num_rows > 0) {
    // If it's already there, just add 1 to the quantity
    $conn->query("UPDATE user_cart SET quantity = quantity + 1 WHERE session_id = '$session_id' AND
product_name = '$product_name'");
} else {
    // If it's not there, insert it as a new row
    $conn->query("INSERT INTO user_cart (session_id, product_name, price, quantity) VALUES ('$session_id',
'$product_name', $price, 1)");
}

// Refresh the page so the user can keep shopping
echo "<script>alert('$product_name added to your cart!'); window.location.href='products.php';</script>";
}
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Aura Fragrances | Shop</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>Our Collection</h1>
        <nav>
            <a href="index.php">HOME</a>
            <a href="cart.php">VIEW CART</a>
        </nav>
    </header>

    <main>
        <section class="category-section">
            <h2 class="category-heading">Signature Perfumes</h2>

            <div class="product-grid">

                <div class="product-card">
                    
                    <h3 class="product-title">Midnight Rose</h3>
                    <p class="product-price">$125.00</p>
                    <form method="POST" action="products.php">
                        <input type="hidden" name="product_name" value="Midnight Rose">
                        <input type="hidden" name="price" value="125.00">
                        <button type="submit" name="add_to_cart" class="auth-btn">ADD TO CART</button>
                    </form>
                </div>

                <div class="product-card">
                    
                    <h3 class="product-title">Oceanic Wood</h3>
                    <p class="product-price">$95.00</p>
                    <form method="POST" action="products.php">
                        <input type="hidden" name="product_name" value="Oceanic Wood">
                        <input type="hidden" name="price" value="95.00">
                        <button type="submit" name="add_to_cart" class="auth-btn">ADD TO CART</button>
                    </form>
                </div>
            </div>
        </section>
    </main>
</body>
</html>

```

```

    </form>
</div>

<div class="product-card">
  
  <h3 class="product-title">Vanilla Silk</h3>
  <p class="product-price">$110.00</p>
  <form method="POST" action="products.php">
    <input type="hidden" name="product_name" value="Vanilla Amber">
    <input type="hidden" name="price" value="110.00">
    <button type="submit" name="add_to_cart" class="auth-btn">ADD TO CART</button>
  </form>
</div>

<div class="product-card">
  
  <h3 class="product-title">Smoked Leather</h3>
  <p class="product-price">$85.00</p>
  <form method="POST" action="products.php">
    <input type="hidden" name="product_name" value="Citrus Bloom">
    <input type="hidden" name="price" value="85.00">
    <button type="submit" name="add_to_cart" class="auth-btn">ADD TO CART</button>
  </form>
</div>

</div>
</section>
</main>

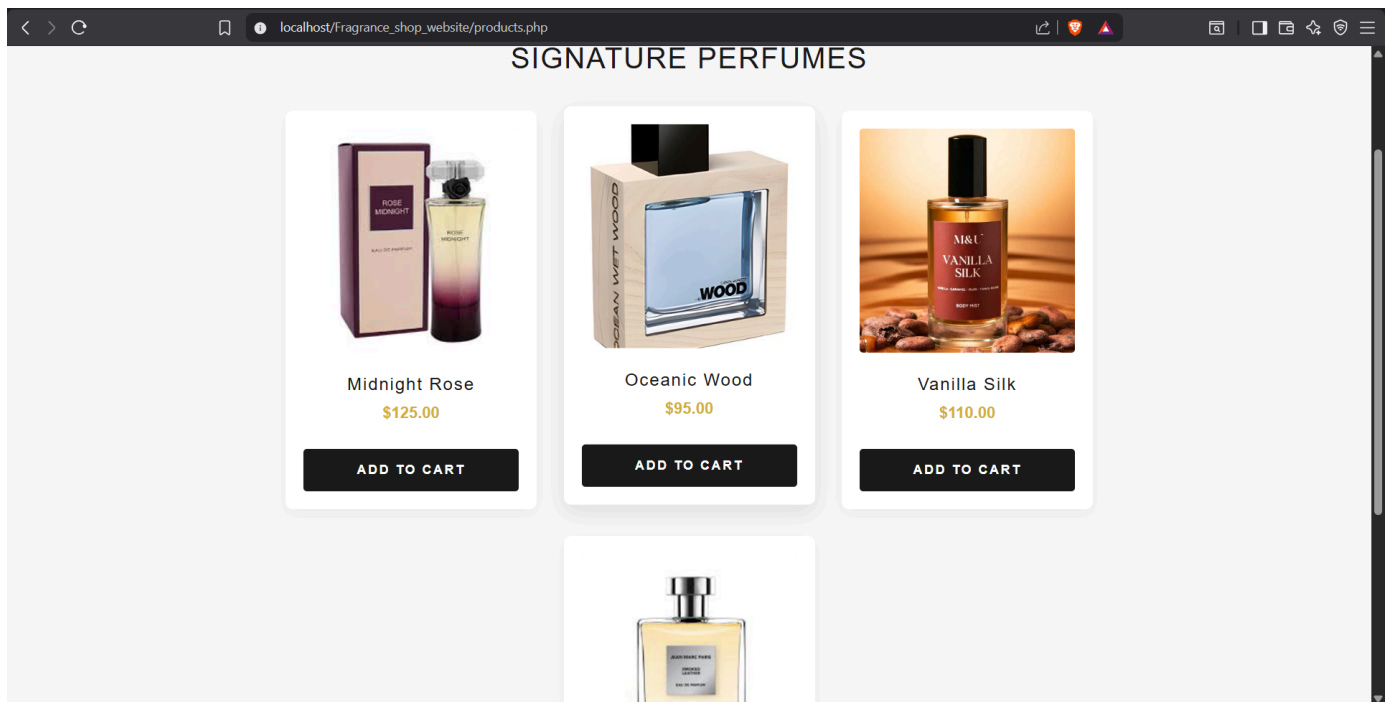
<footer>
  <p>&copy; 2026 Aura Fragrances.</p>
</footer>

<script src="script.js"></script>
</body>
</html>

```

Output:

A. menu/product page output:



Code:

B. cart page:

Code:

```
<?php
session_start();
require 'db.php';
$session_id = session_id();

// Handle Remove Item
if (isset($_GET['remove'])) {
    $cart_id = (int)$_GET['remove'];
    $conn->query("DELETE FROM user_cart WHERE cart_id = $cart_id AND session_id = '$session_id'");
    header("Location: cart.php");
    exit();
}

// Fetch Cart Items
$cart_items = $conn->query("SELECT * FROM user_cart WHERE session_id = '$session_id'");
$grand_total = 0;
?>
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Aura Fragrances | Cart</title>
    <link rel="stylesheet" href="styles.css">
</head>
<body>
    <header>
        <h1>Your Shopping Cart</h1>
        <nav><a href="index.php">HOME</a> | <a href="products.php">SHOP</a></nav>
    </header>
```

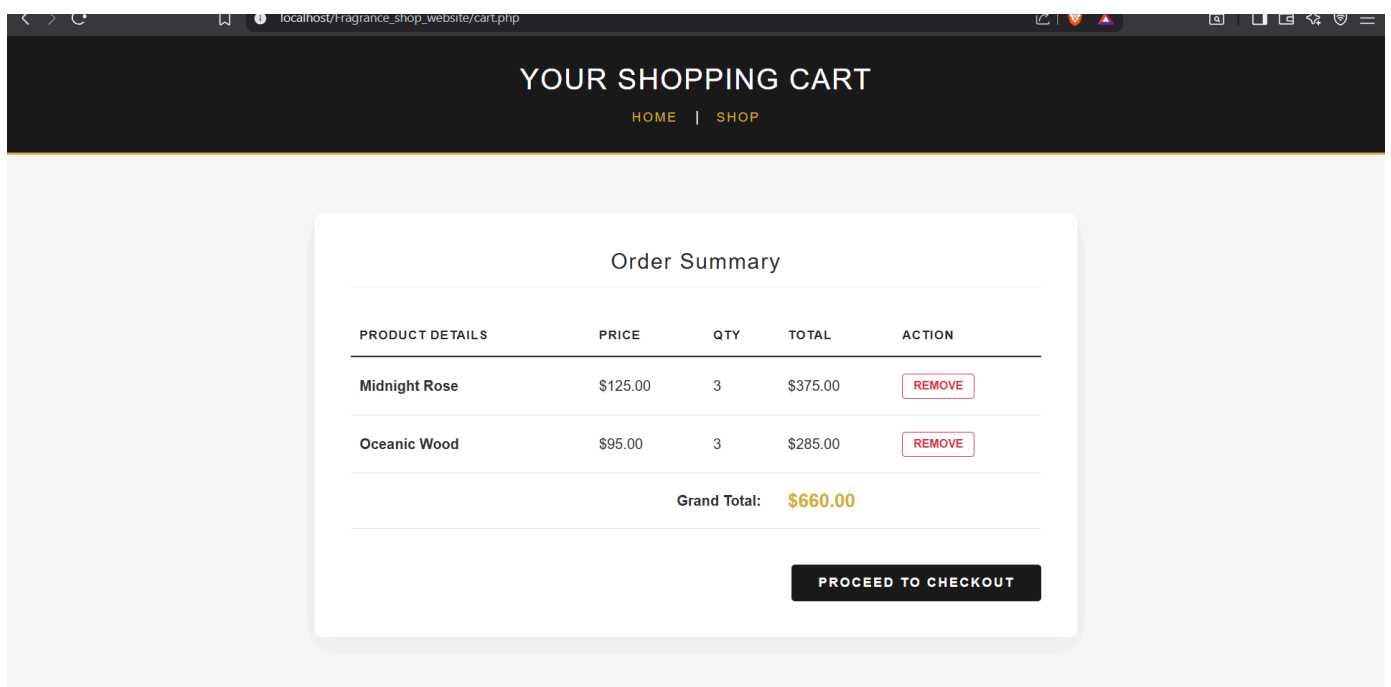
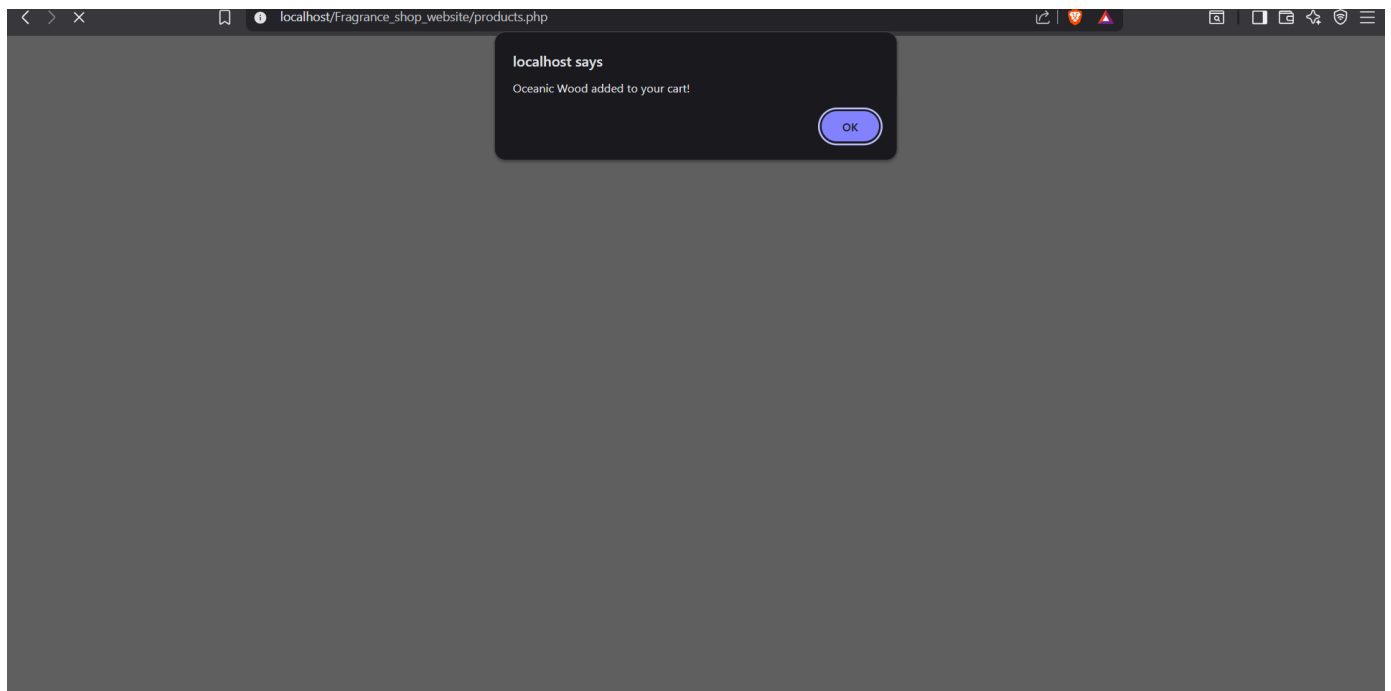
```

<main class="auth-main">
  <div class="auth-card" style="max-width: 850px;">
    <h2>Order Summary</h2>
    <div class="table-responsive">
      <table class="luxury-table">
        <thead>
          <tr>
            <th>Product Details</th>
            <th>Price</th>
            <th>Qty</th>
            <th>Total</th>
            <th>Action</th>
          </tr>
        </thead>
        <tbody>
          <?php if ($cart_items->num_rows > 0): ?>
            <?php while($item = $cart_items->fetch_assoc()):
              $total = $item['price'] * $item['quantity'];
              $grand_total += $total;
            ?>
            <tr>
              <td><strong><?= $item['product_name'] ?></strong></td>
              <td><?= number_format($item['price'], 2) ?></td>
              <td><?= $item['quantity'] ?></td>
              <td><?= number_format($total, 2) ?></td>
              <td>
                <a href="cart.php?remove=<?= $item['cart_id'] ?>" class="btn-remove"
style="text-decoration:none;">REMOVE</a>
              </td>
            </tr>
          <?php endwhile; ?>
          <?php else: ?>
            <tr><td colspan="5" style="text-align:center; padding: 30px;">Your cart is empty.</td></tr>
          <?php endif; ?>
        </tbody>
        <tfoot>
          <tr>
            <td colspan="3" class="text-right"><strong>Grand Total:</strong></td>
            <td colspan="2"><strong style="color: #d4af37; font-size: 1.3rem;"><?=
number_format($grand_total, 2) ?></strong></td>
          </tr>
        </tfoot>
      </table>
    </div>
    <?php if ($grand_total > 0): ?>
      <div style="text-align: right; margin-top: 30px;">
        <button class="auth-btn" style="width: auto; padding: 12px 30px;">PROCEED TO
CHECKOUT</button>
      </div>
    <?php endif; ?>
  </div>
</main>
<script src="script.js"></script>
</body>
</html>

```

Output:

B. cart page output:



The screenshot shows the phpMyAdmin interface for a database named 'aura_fragrance'. The 'user_cart' table is selected, and the SQL query 'SELECT * FROM `user\_cart`' has been executed. The results show two rows of data:

cart_id	session_id	product_name	price	quantity
4	m0tobus53sgpkauhi7ap6rmk01	Midnight Rose	125.00	3
5	m0tobus53sgpkauhi7ap6rmk01	Oceanic Wood	95.00	3

The interface includes a sidebar with a database structure tree, a top navigation bar with various tools (Browse, Structure, SQL, Search, Insert, Export, Import, Privileges, Operations, Tracking, Triggers), and a bottom console area.

Conclusion:

In this experiment, the e-commerce architecture was significantly upgraded by migrating the ephemeral client-side shopping cart to a persistent, server-side Relational Database Management System (MySQL). Through the use of the `mysqli` extension, the PHP script successfully executed SQL Data Manipulation Language (DML) commands—`INSERT`, `SELECT`, `UPDATE`, and `DELETE`—to manage the cart state dynamically. By utilizing the PHP Session ID as a foreign key to track anonymous user carts, the application demonstrated a robust, enterprise-level approach to state management that ensures data consistency and persistence independent of the user's local browser environment.

Experiment No.10

Problem Statement:

- Develop a PHP script to handle the checkout process for users who are ready to complete their purchase. The script should process the cart data and provide feedback to the user upon successful or failed checkout.
- Develop a PHP script that processes the checkout process for users who are ready to complete their purchase, integrating the MySQL database for handling user and order information. The script should validate user input, process the cart data, and provide feedback upon successful or failed checkout.

Objective:

To construct the backend checkout logic using PHP and MySQL to process cart data, generate an order record, and finalize the e-commerce transaction.

Theory:

Deep Theoretical Background: While `localStorage` provides temporary client-side persistence, true enterprise applications require central, server-side data persistence. This is achieved by integrating PHP with an RDBMS like **MySQL**.

1. Relational Theory and Normalization: MySQL stores data in tables consisting of rows and columns, governed by relational algebra. The `user_cart` table is an entity designed to hold ephemeral data. It requires a schema: `id` (Primary Key), `session_id` (Foreign Key proxy to identify the user), `product_name`, `price`, and `quantity`.

2. The SQL Communication Layer: PHP cannot speak directly to the database files; it uses an API extension called `mysqli` (MySQL Improved) to send **SQL (Structured Query Language)** commands.

- **Data Manipulation Language (DML):** The script executes `SELECT` statements to check for existing cart items. Based on the result matrix, it branches logically: if the item exists, it executes an `UPDATE` statement to increment the quantity; if it is new, it executes an `INSERT` statement to append a new row.
- **Protection against SQL Injection (SQLi):** If user input is concatenated directly into an SQL string, an attacker can input malicious SQL syntax (e.g., `' ; DROP TABLE users; --`). To mitigate this, data must be passed through `$conn->real_escape_string()`, which neutralizes syntactic control characters, or better, utilizing Prepared Statements where the data and the SQL logic are compiled separately.

Code:

A. Checkout page:

Code:

```
<?php
session_start();
require 'db.php';
```



```

// 1. Check if user is logged in
if (!isset($_SESSION['logged_in'])) {
    echo "<script>alert('Please log in to finalize your purchase.');

```

```

    <nav><a href="cart.php">RETURN TO CART</a></nav>
</header>

<main class="auth-main">
  <div class="auth-card">
    <h2>Order Review</h2>
    <div style="text-align: left; margin: 20px 0; border-top: 1px solid #eee; padding-top: 20px;">
      <p><strong>Account:</strong> <?= $user_email ?></p>
      <p><strong>Total Amount:</strong> <span style="color: #d4af37; font-weight: bold;">$<?=
number_format($grand_total, 2) ?></span></p>
    </div>

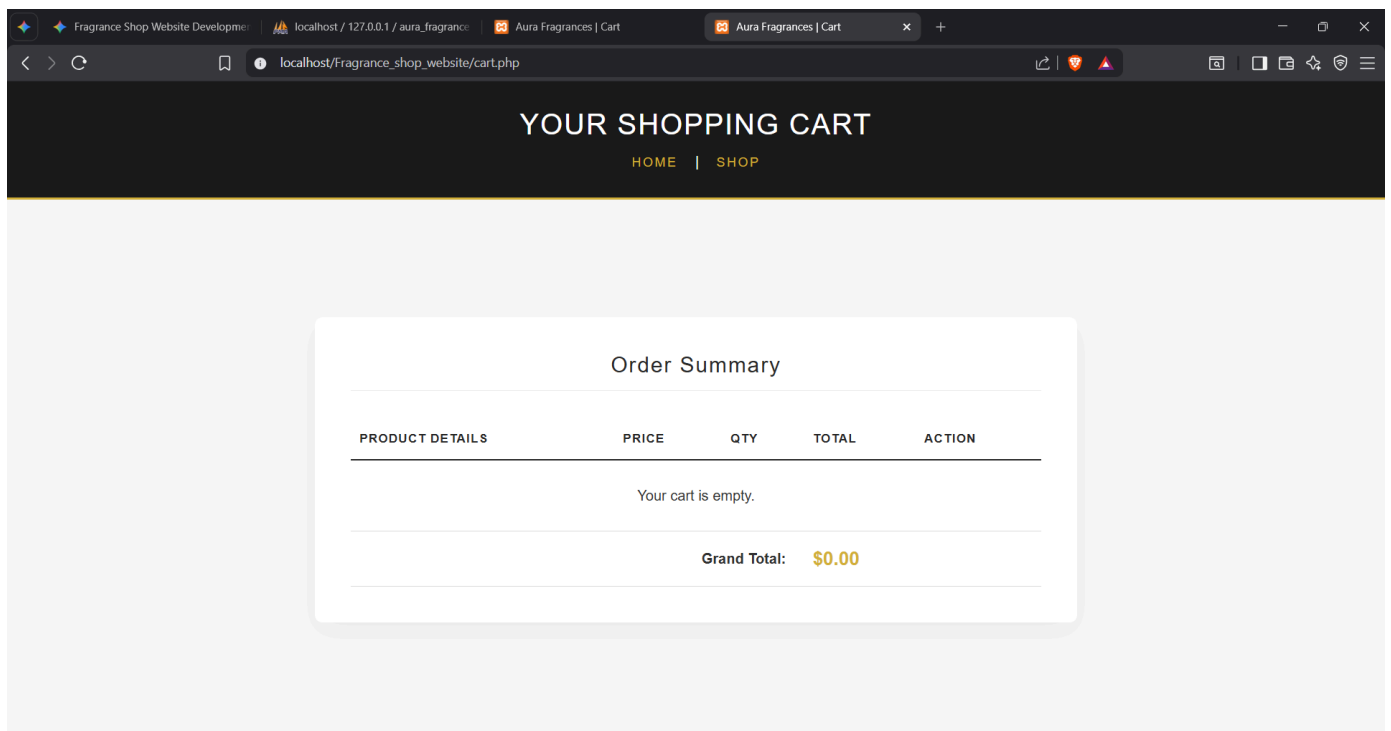
    <form action="checkout.php" method="POST">
      <button type="submit" class="auth-btn">CONFIRM ORDER</button>
    </form>
  </div>
</main>

<footer>
  <p>&copy; 2026 Aura Fragrances.</p>
</footer>
</body>
</html>

```

Output:

A. Checkout page output:



The screenshot shows the phpMyAdmin web interface. The left sidebar displays the database structure with 'aura_fragrance' selected. The main panel shows the 'orders' table with the following data:

order_id	user_email	total_amount	order_date
1	vaidehi@test.com	330.00	2026-05-05 00:33:09

The interface also includes a 'Query results operations' section with options like 'Print', 'Copy to clipboard', 'Export', 'Display chart', and 'Create view'. A 'Bookmark this SQL query' section is visible at the bottom.

Conclusion:

This final experiment successfully culminated the e-commerce pipeline by architecting a secure backend checkout process. The script demonstrated the "Zero-Trust" security principle by completely ignoring client-side calculations and instead utilizing SQL aggregate functions (**SUM**) to calculate the definitive financial total directly on the database server. The successful execution of a multi-step transaction—verifying cart contents, writing the final data to a persistent **orders** ledger, retrieving the auto-generated primary key, and subsequently purging the temporary **user_cart** table—highlighted the critical importance of atomic operations in software engineering to maintain strict financial and relational data integrity.